

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

**Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники**

**Отчет по лабораторной работе №3
по дисциплине «Искусственный интеллект и машинное обучение»**

Выполнил студент Ромащенко Данил группы ИВТ-б-о-24-1

Подпись студента _____
Работа защищена « » _____ 20__ г.
Проверил Воронкин Р.А. _____
(подпись)

Ставрополь 2026

Тема: «Основы работы с библиотекой matplotlib».

Цель работы: исследовать базовые возможности библиотеки matplotlib языка программирования Python.

Ход работы

Вариант 17

Ссылка на репозиторий: https://github.com/I1N322/lab_3.git

1) Был создан общедоступный репозиторий, в котором использована лицензия MIT и язык программирование Python.

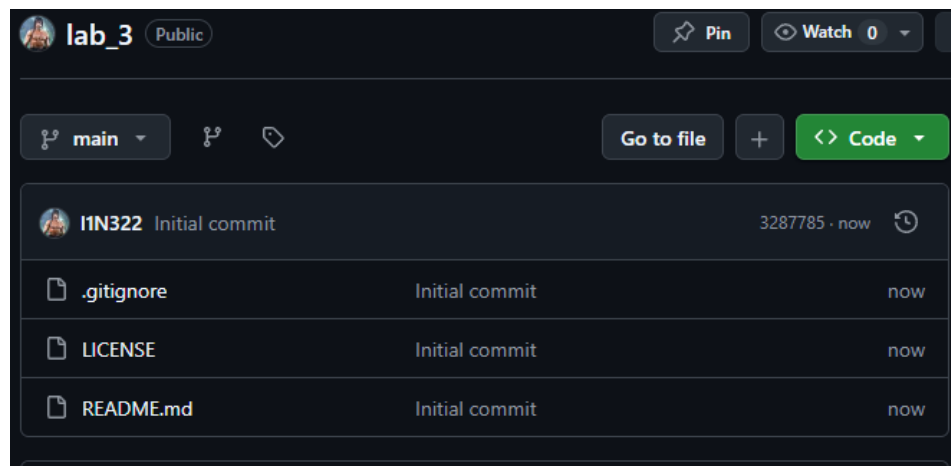


Рисунок 1. Созданный репозиторий

2) Был дополнен файл .gitignore необходимыми правилами.

```
220 # Covers JetBrains IDEs: IntelliJ, GoLand, RubyMine, PhpStorm, AppCode, PyCharm, CLion, Android Studio, WebStorm and Rider
221 # Reference: https://intellij-support.jetbrains.com/hc/en-us/articles/206544839
222
223 # User-specific stuff
224 .idea/**/workspace.xml
225 .idea/**/tasks.xml
226 .idea/**/usage.statistics.xml
227 .idea/**/dictionaries
228 .idea/**/shelf
229
230 # AWS User-specific
231 .idea/**/aws.xml
232
233 # Generated files
234 .idea/**/contentModel.xml
235
236 # Sensitive or high-churn files
237 .idea/**/dataSources/
238 .idea/**/dataSources.ids
239 .idea/**/dataSources.local.xml
240 .idea/**/sqlDataSources.xml
241 .idea/**/dynamic.xml
242 .idea/**/uiDesigner.xml
243 .idea/**/dbnavigator.xml
244
245 # Gradle
246 .idea/**/gradle.xml
247 .idea/**/libraries
248
249 # Gradle and Maven with auto-import
250 # When using Gradle or Maven with auto-import, you should exclude module files,
251 # since they will be recreated, and may cause churn. Uncomment if using
252 # auto-import.
253 # .idea/artifacts
254 # .idea/compiler.xml
```

Рисунок 2. Добавленные правила в файл .gitignore

3) Было выполнено клонирование созданного репозитория на компьютер.

```
USER@C0RPG-MINOW04 ~$ git clone https://github.com/11N322/lab_3.git
Cloning into 'lab_3'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (8/8), 5.20 KiB | 5.20 MiB/s, done.
Resolving deltas: 100% (1/1), done.
```

Рисунок 3. Клонирование репозитория

4) Были проработаны примеры.

```
import matplotlib.pyplot as plt
import numpy as np
%matplotlib inline

x = np.linspace(0, 10, 50)
y = x

plt.title("Линейная зависимость y = x")
plt.xlabel("x")
plt.ylabel("y")
plt.grid()

plt.plot(x, y, "r--")
```

[<matplotlib.lines.Line2D at 0x1d15dc02d50>]

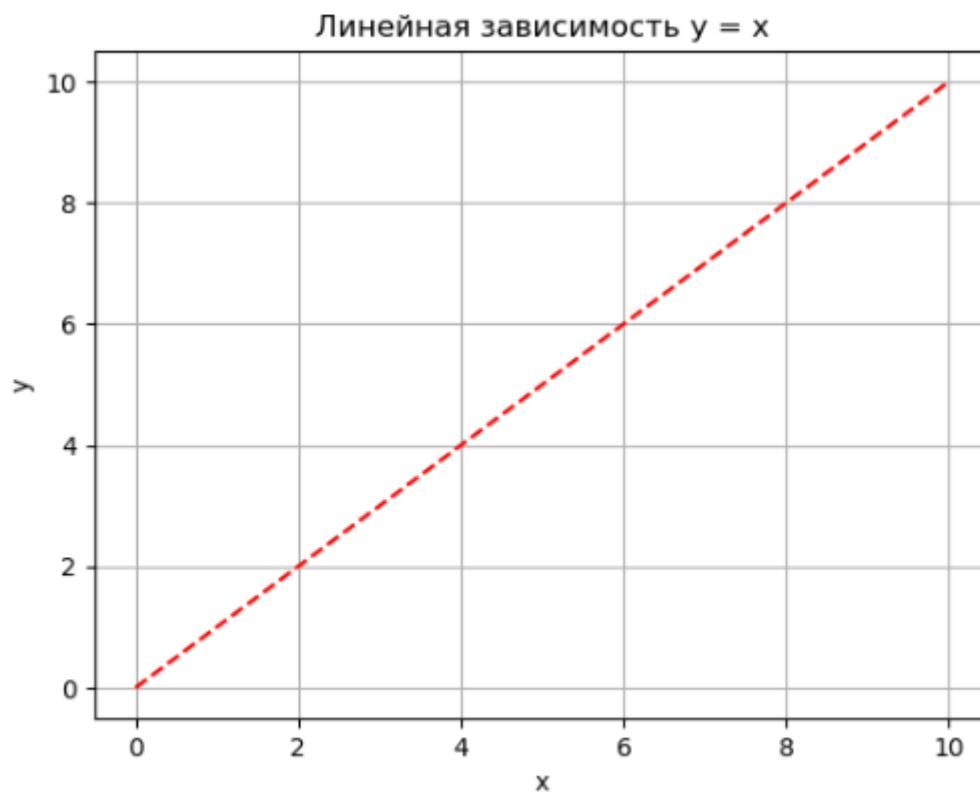


Рисунок 4. Пример 1

```

x = np.linspace(0, 10, 50)
y1 = x

y2 = [i**2 for i in x]

plt.title("Зависимости: y1 = x, y2 = x^2")
plt.xlabel("x")
plt.ylabel("y1, y2")
plt.grid()

plt.plot(x, y1, x, y2)

```

```

[<matplotlib.lines.Line2D at 0x1d15df02e90>,
 <matplotlib.lines.Line2D at 0x1d15df02fd0>]

```

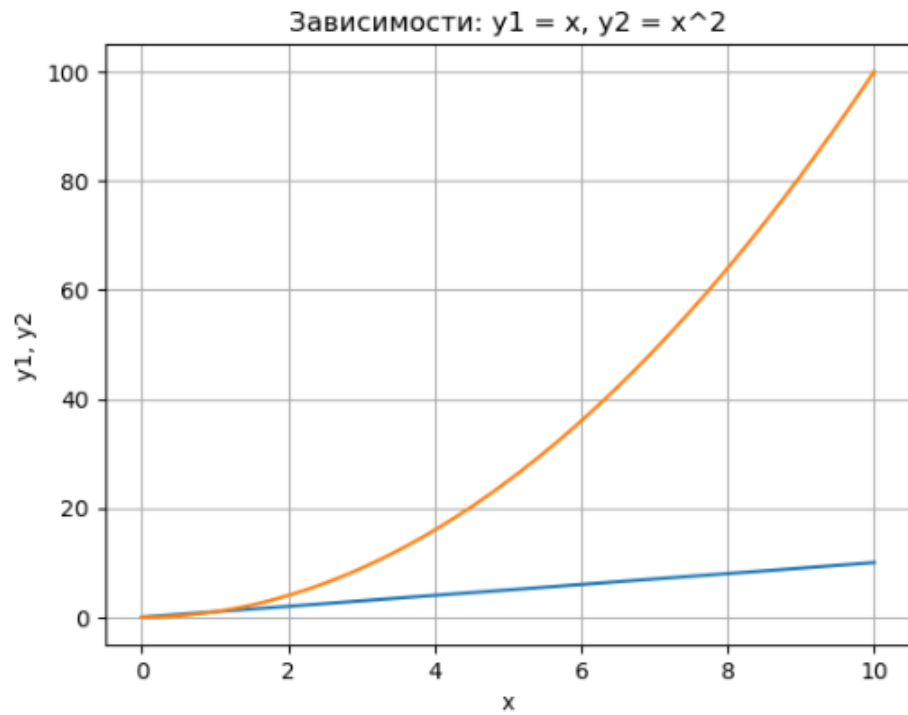


Рисунок 5. Пример 2

```
plt.figure(figsize=(9, 9))

plt.subplot(2, 1, 1)
plt.plot(x, y1)

plt.title("Зависимости:  $y_1 = x$ ,  $y_2 = x^2$ ")
plt.ylabel("y1", fontsize = 14)
plt.grid(True)

plt.subplot(2, 1, 2)
plt.plot(x, y2)

plt.xlabel("x", fontsize = 14)
plt.ylabel("y2", fontsize = 14)
plt.grid(True)
```

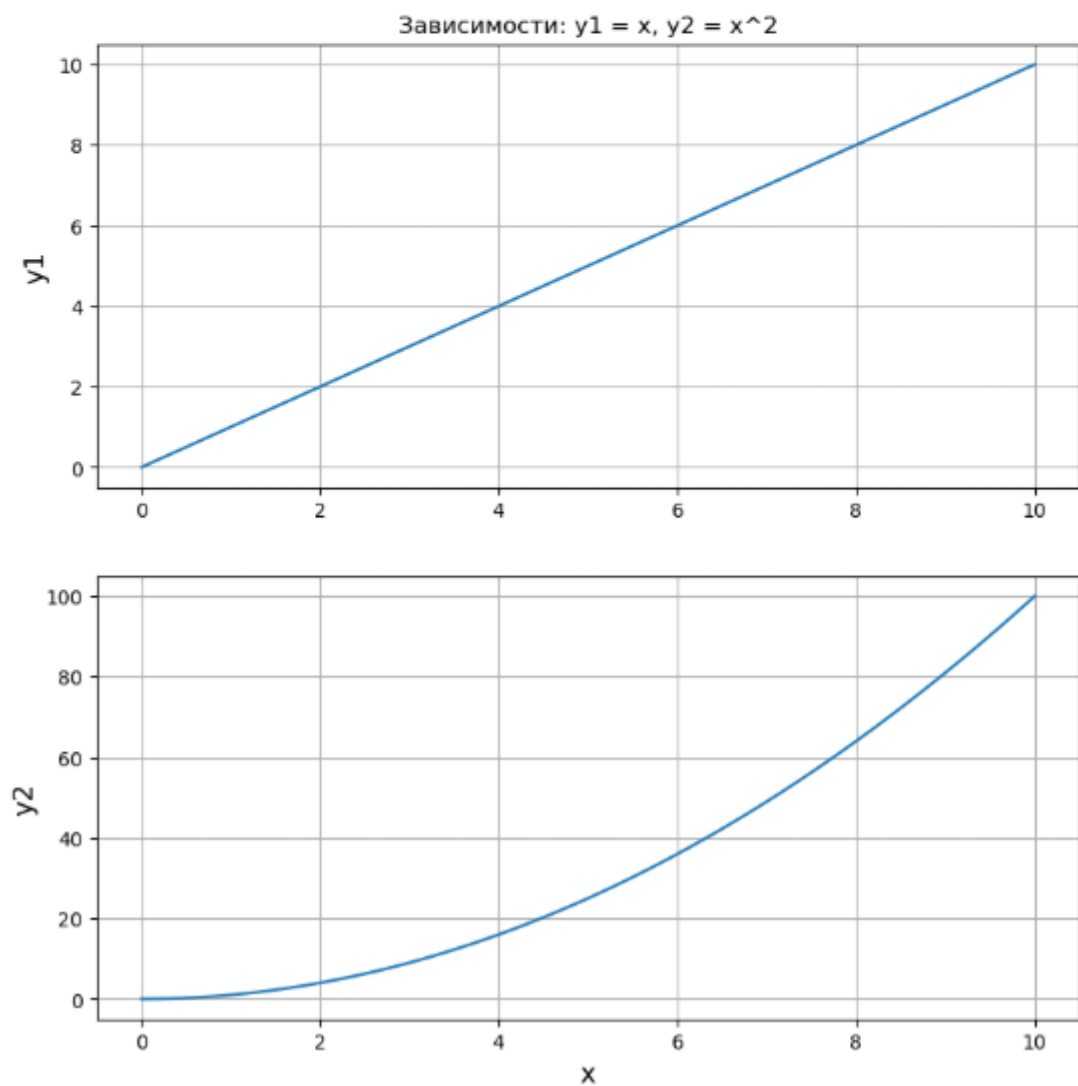


Рисунок 6. Пример 3

```

fruits = ["apple", "peach", "orange", "banana", "melon"]
counts = [34, 25, 43, 31, 17]

plt.bar(fruits, counts)
plt.title("Fruits!")
plt.xlabel("Fruit")
plt.ylabel("Count")

```

```
Text(0, 0.5, 'Count')
```

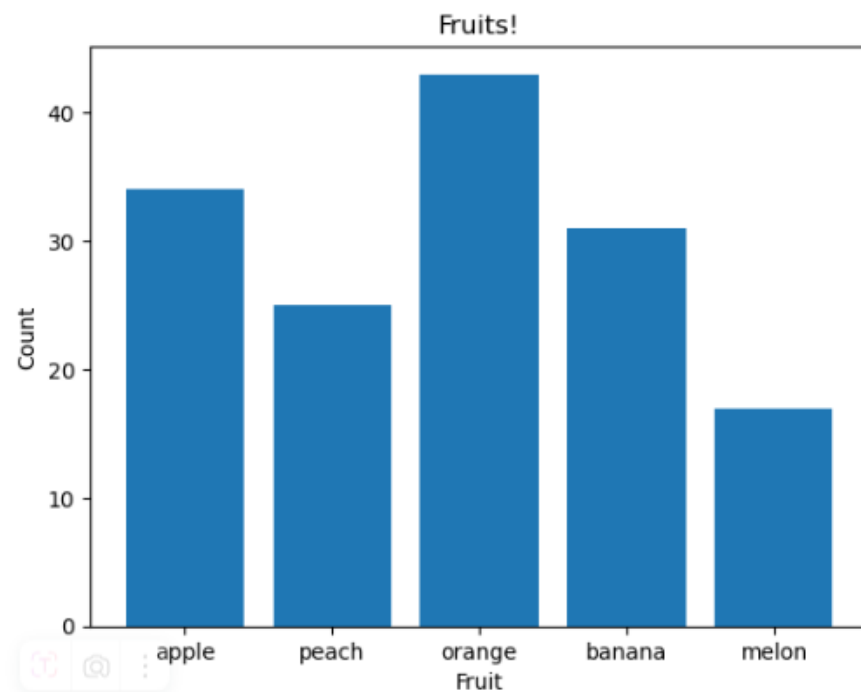


Рисунок 7. Пример 4

```
plt.plot([1, 7, 3, 5, 11, 1])
```

```
[<matplotlib.lines.Line2D at 0x1d15dfd4f50>]
```

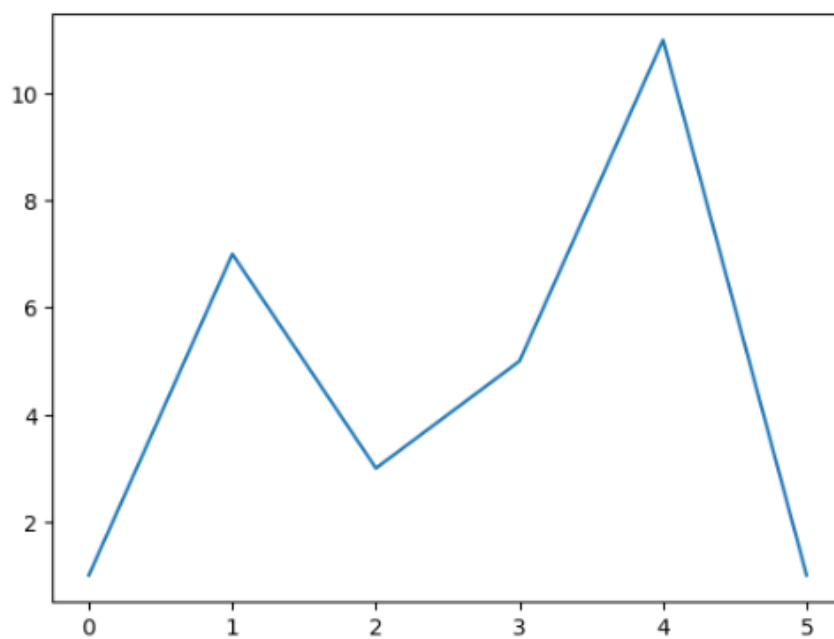


Рисунок 8. Пример 5

```

x = [1, 5, 10, 15, 20]
y = [1, 7, 3, 5, 11]

plt.plot(x, y, label = "steel price")
plt.title("Chart price", fontsize = 15)
plt.xlabel("Day", fontsize = 12, color = "blue")
plt.ylabel("Price", fontsize = 12, color = "blue")

plt.legend()
plt.grid(True)

plt.text(15, 4, "grow up!")

```

```
Text(15, 4, 'grow up!')
```

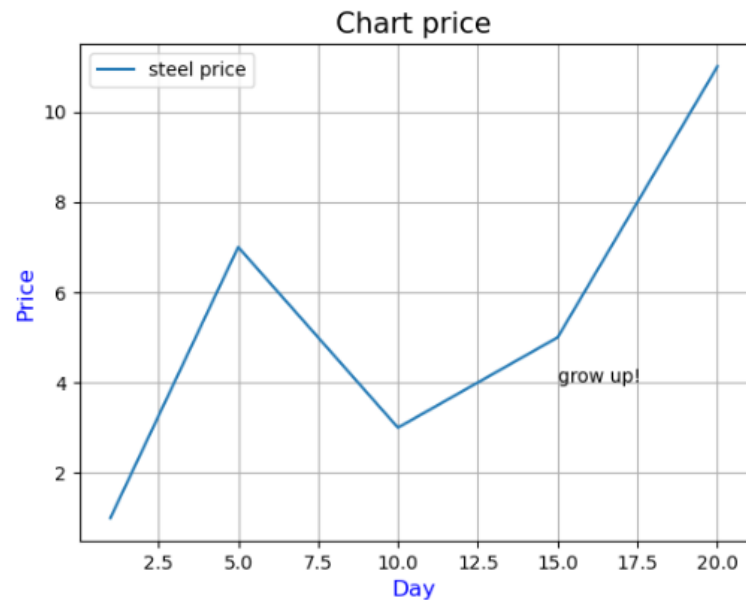


Рисунок 9. Пример 6

```

line = plt.plot(x, y)
plt.setp(line, linestyle = '--')

```

```
[None]
```

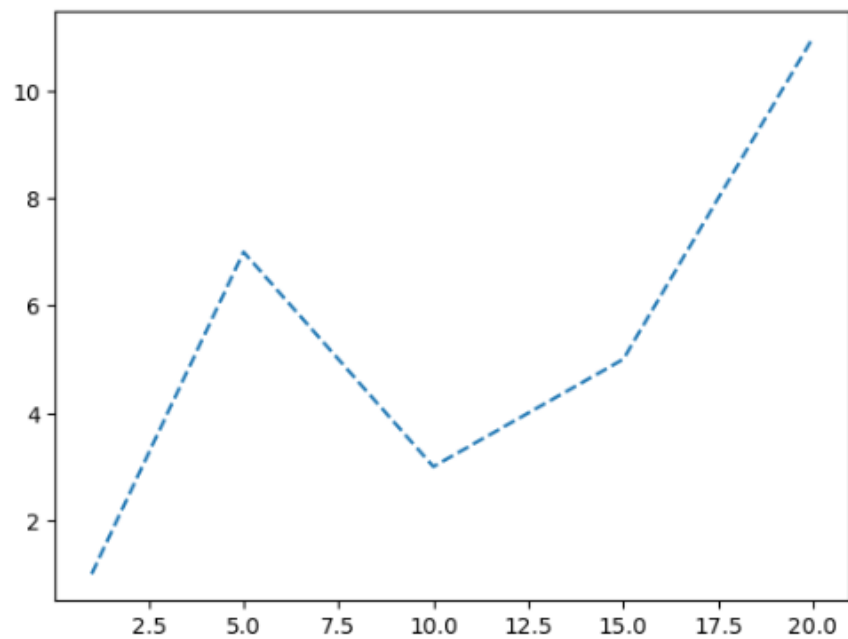


Рисунок 10. Пример 7

```

y1 = y
y2 = [i*1.2 + 1 for i in y1]
y3 = [i*1.2 + 1 for i in y2]
y4 = [i*1.2 + 1 for i in y3]

plt.plot(x, y1, '-', x, y2, '--', x, y3, '-.', x, y4, ':')

```

```

[<matplotlib.lines.Line2D at 0x1d15f557b10>,
 <matplotlib.lines.Line2D at 0x1d15f557c50>,
 <matplotlib.lines.Line2D at 0x1d15f557d90>,
 <matplotlib.lines.Line2D at 0x1d15f557ed0>]

```

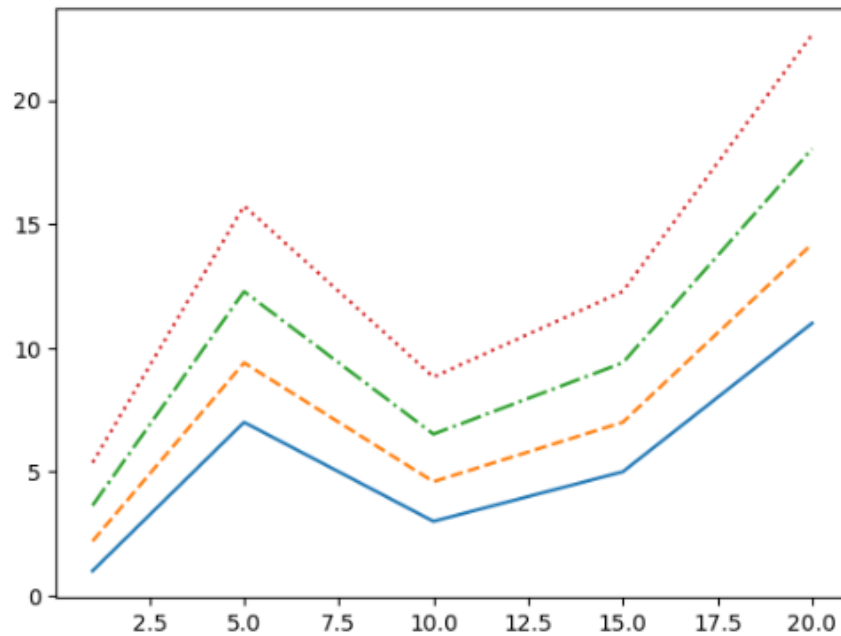


Рисунок 11. Пример 8

```

plt.plot(x, y, '--r')

```

```

[<matplotlib.lines.Line2D at 0x1d15ddd490>]

```

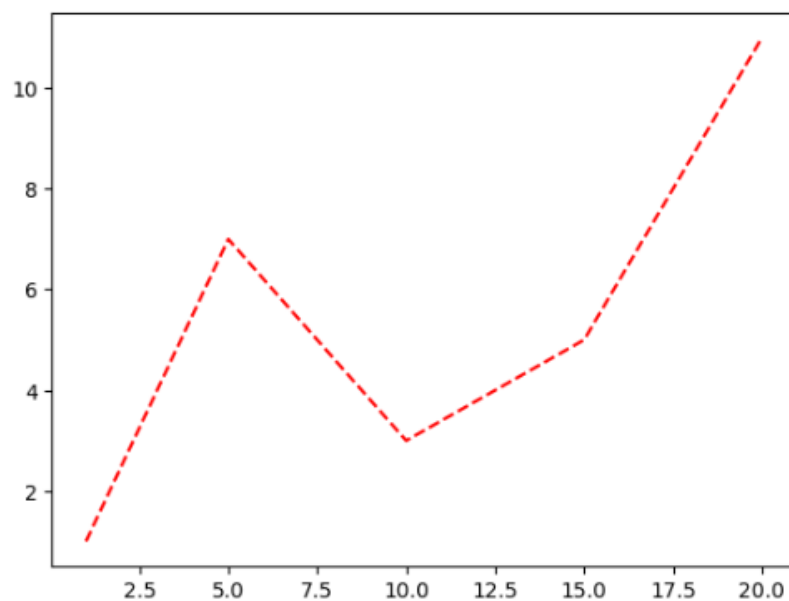


Рисунок 12. Пример 9


```
plt.plot(x, y, 'ro')
```

```
[<matplotlib.lines.Line2D at 0x1d15f5c0a50>]
```

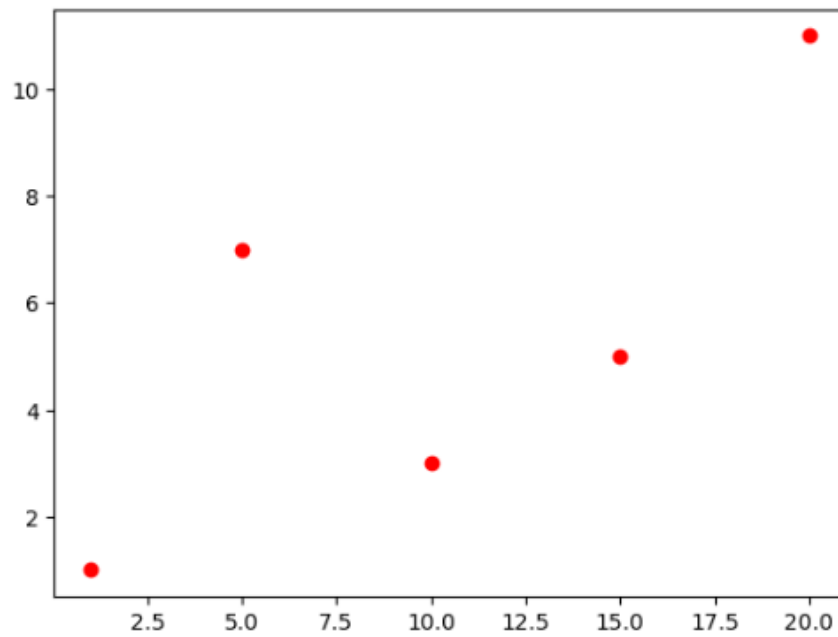


Рисунок 13. Пример 10

```
plt.plot(x, y, 'bx')
```

```
[<matplotlib.lines.Line2D at 0x1d15e20afd0>]
```

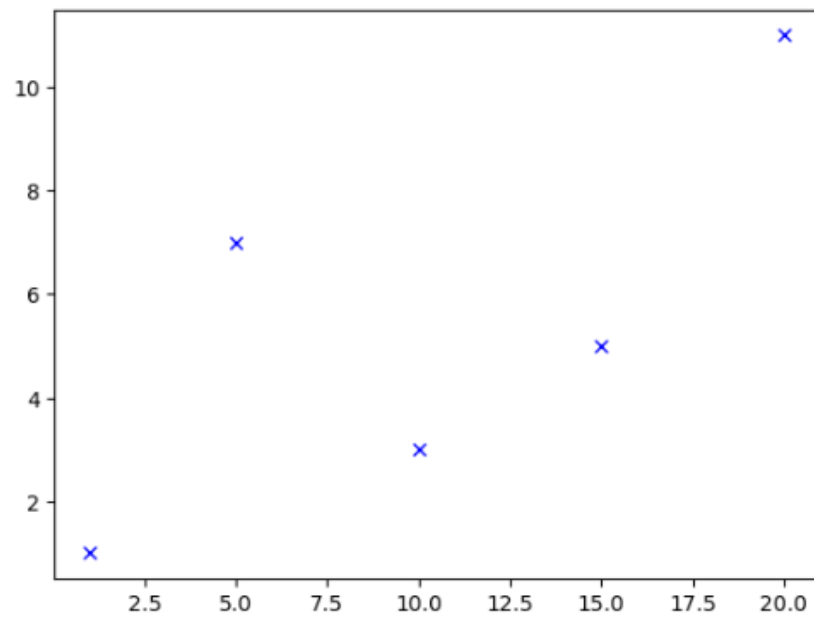


Рисунок 14. Пример 11

```
plt.figure(figsize=(12, 7))
```

```
plt.subplot(2, 2, 1)
plt.plot(x, y1, '-')
```

```
plt.subplot(2, 2, 2)
plt.plot(x, y2, '-')
```

```
plt.subplot(2, 2, 3)
plt.plot(x, y3, '-')
```

```
plt.subplot(2, 2, 4)
plt.plot(x, y4, ':')
```

```
[<matplotlib.lines.Line2D at 0x1d15e0c2c10>]
```

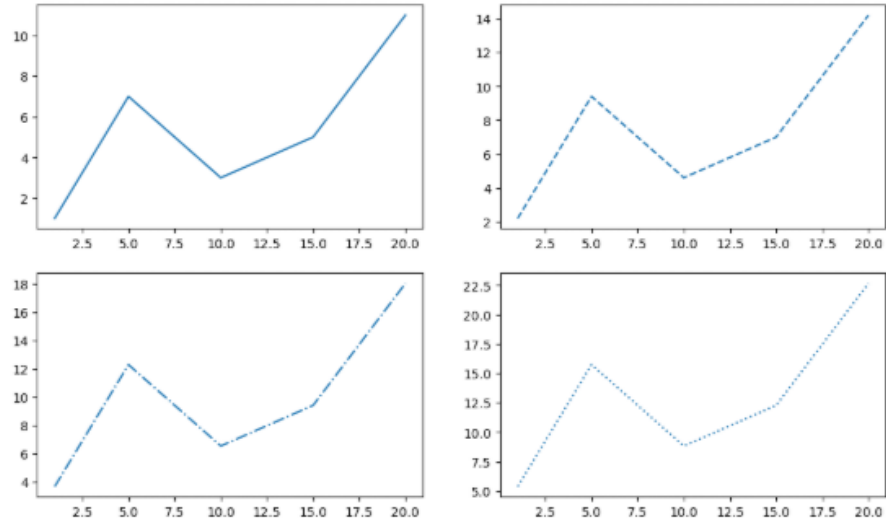


Рисунок 15. Пример 12

```
x = [1, 5, 10, 15, 20]
y1 = [1, 7, 3, 5, 11]
y2 = [4, 3, 1, 8, 12]
```

```
plt.figure(figsize = (12, 7))
plt.plot(x, y1, 'o-r', alpha=0.7, label='first', lw=5, mec='b', mew=2, ms=10)
plt.plot(x, y2, 'v-.g', label='second', mec='r', lw=2, mew=2, ms=12)

plt.legend()
plt.grid(True)
```

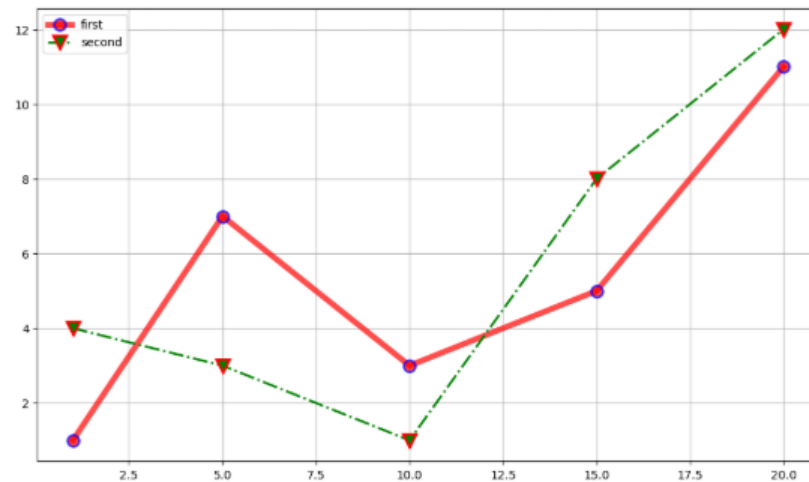


Рисунок 16. Пример 13

```
x = np.arange(0.0, 5, 0.01)
y = np.cos(x*np.pi)

plt.plot(x, y, c = 'r')
plt.fill_between(x, y)
```

<matplotlib.collections.FillBetweenPolyCollection at 0x1d15de917f0>

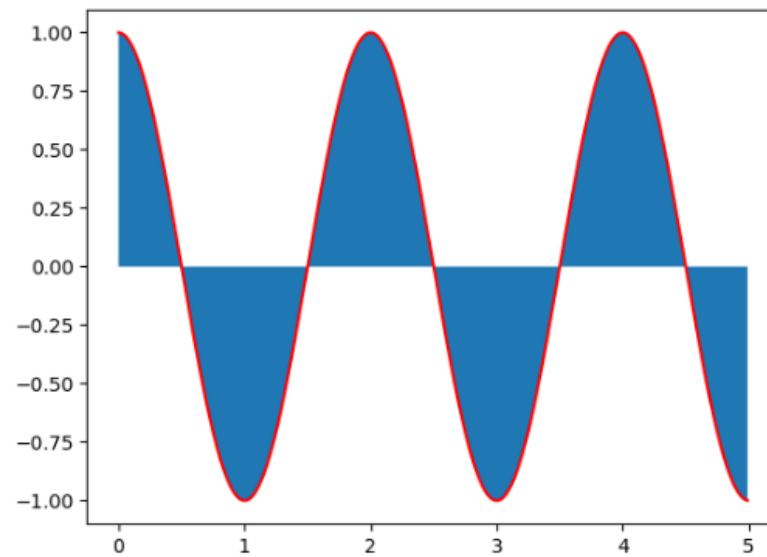


Рисунок 17. Пример 14

```
plt.plot(x, y, c = 'r')
plt.fill_between(x, y, where=(y > 0.75) | (y < -0.75))
```

<matplotlib.collections.FillBetweenPolyCollection at 0x1d160182d50>

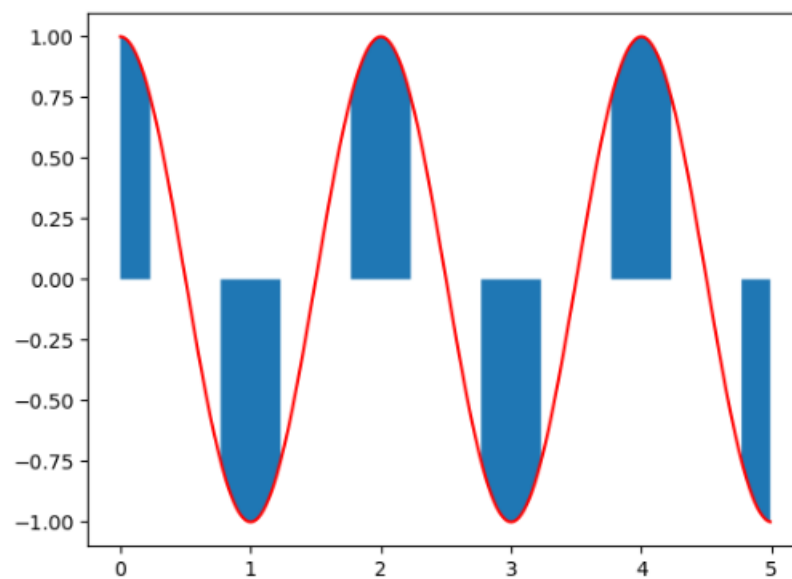


Рисунок 18. Пример 15

```
plt.plot(x, y, c = "r")
plt.grid()

plt.fill_between(x, y, where = y >= 0, color = "g", alpha = 0.3)
plt.fill_between(x, y, where = y <= 0, color = "r", alpha = 0.3)
```

<matplotlib.collections.FillBetweenPolyCollection at 0x1d15f71e5d0>

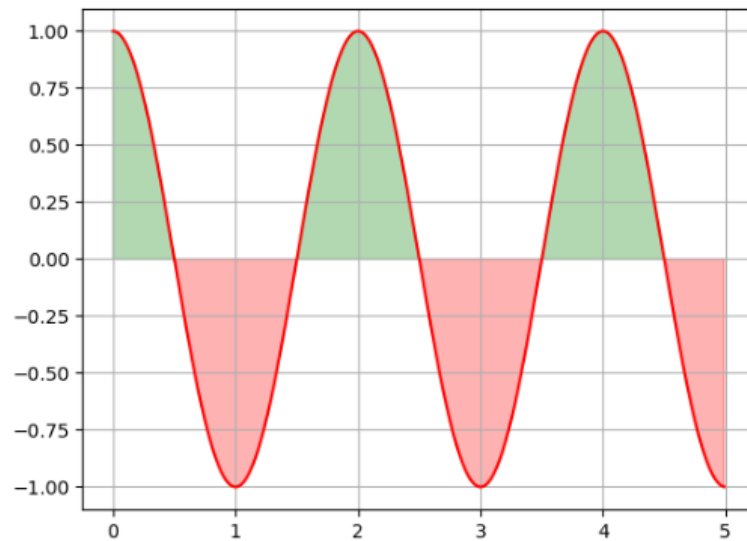


Рисунок 19. Пример 16

```
x = [1, 2, 3, 4, 5, 6, 7]
y = [7, 6, 5, 4, 5, 6, 7]

plt.plot(x, y, marker="o", c="g")
```

[<matplotlib.lines.Line2D at 0x1d15f7b5950>]

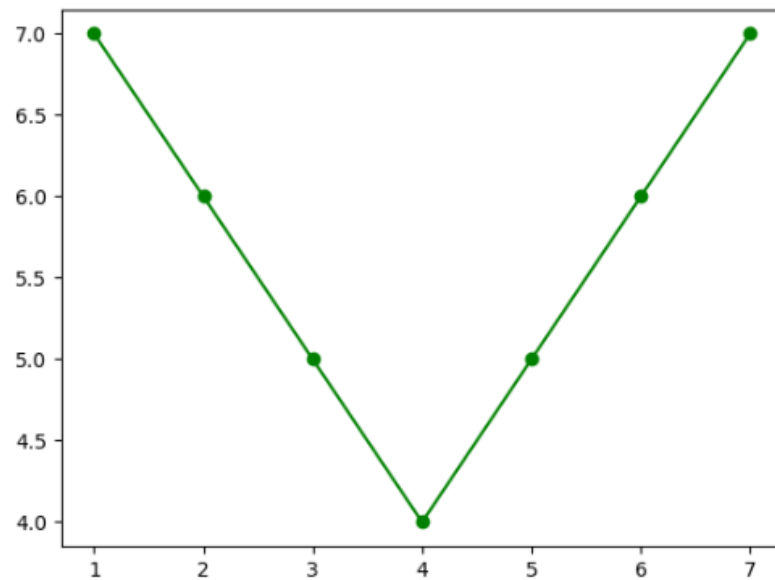


Рисунок 20. Пример 17

```

x = np.arange(0.0, 5, 0.01)

y = np.cos(x * np.pi)

m_ev_case = [None, 10, (100, 30), slice(100, 400, 15), [0, 100, 200, 300],
             [10, 50, 100]]

fig, ax = plt.subplots(2, 3, figsize=(10, 7))
axs = [ax[i, j] for i in range(2) for j in range(3)]

for i, case in enumerate(m_ev_case):
    axs[i].set_title(str(case))
    axs[i].plot(x, y, "o", ls='-', ms=7, markevery=case)

```

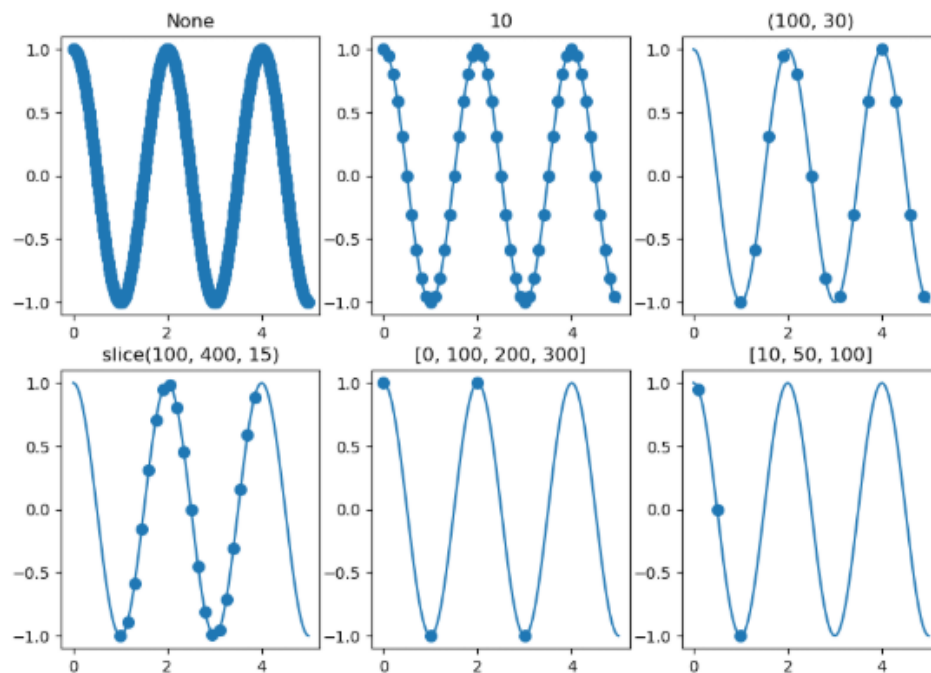


Рисунок 21. Пример 18

```

y_masked = np.ma.masked_where(y < -0.5, y)
plt.ylim(-1, 1)

plt.plot(x, y_masked, linewidth=3)

```

[<matplotlib.lines.Line2D at 0x1d15f8974d0>]

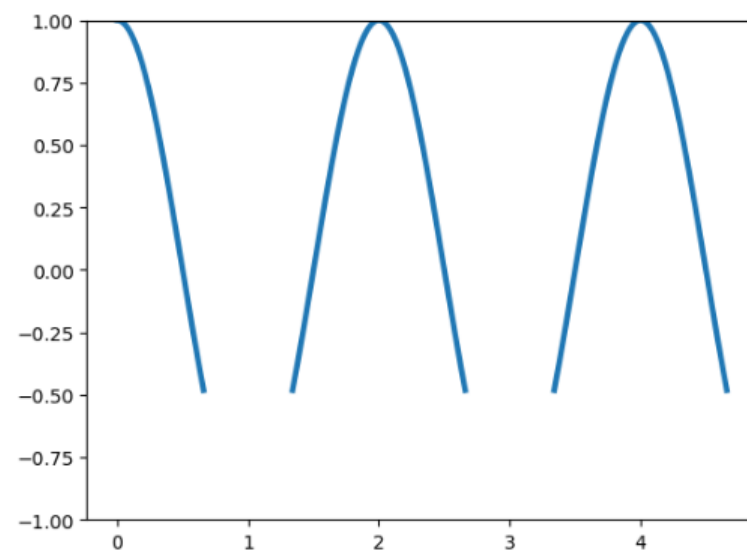


Рисунок 22. Пример 19

```

x = np.arange(0, 7)
y = x

where_set = ['pre', 'post', 'mid']
fig, axs = plt.subplots(1, 3, figsize=(15, 4))

for i, ax in enumerate(axs):
    ax.step(x, y, "g-o", where=where_set[i])
    ax.grid()

```

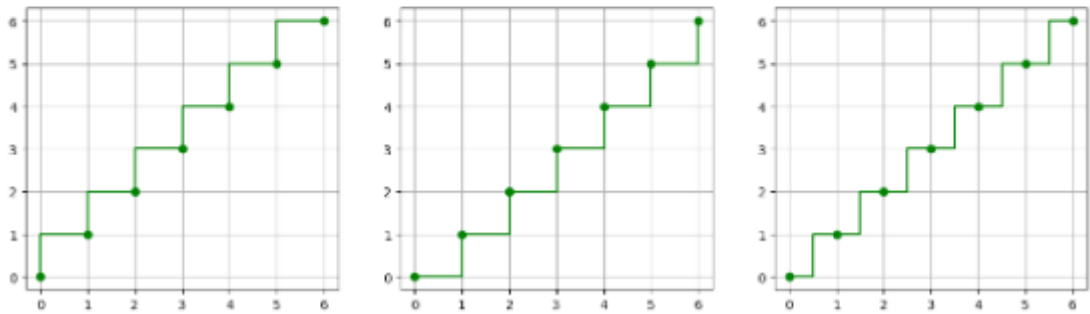


Рисунок 23. Пример 20

```

x = np.arange(0, 11, 1)

y1 = np.array([(-0.2)*i**2+2*i for i in x])
y2 = np.array([(-0.4)*i**2+4*i for i in x])
y3 = np.array([2*i for i in x])

labels = ["y1", "y2", "y3"]

fig, ax = plt.subplots()

ax.stackplot(x, y1, y2, y3, labels=labels)
ax.legend(loc="upper left")

```

<matplotlib.legend.Legend at 0x1d160269e50>

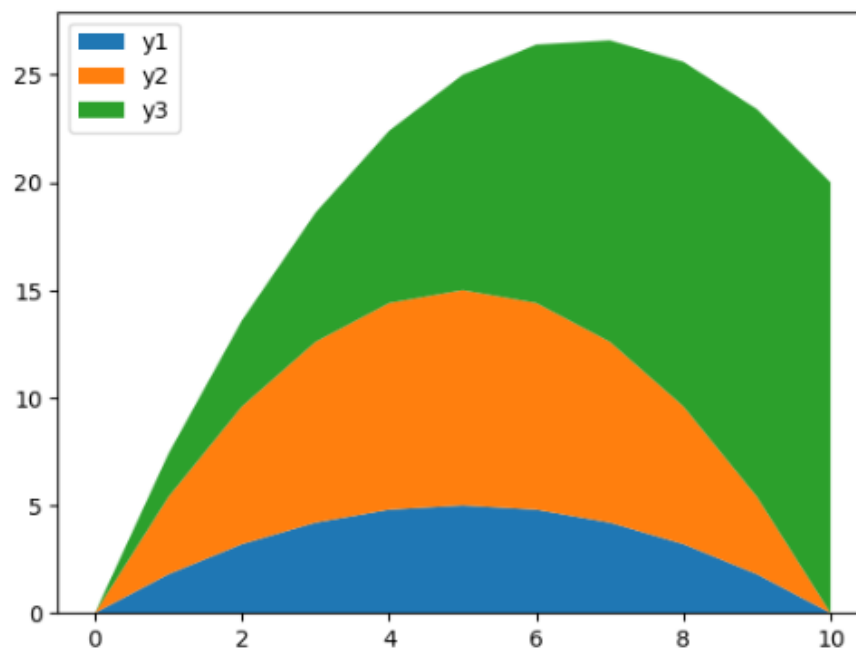


Рисунок 24. Пример 21

```
x = np.arange(0, 10.5, 0.5)
y = np.array([(-0.2)*i**2+2*i for i in x])

plt.stem(x, y)
```

<StemContainer object of 3 artists>

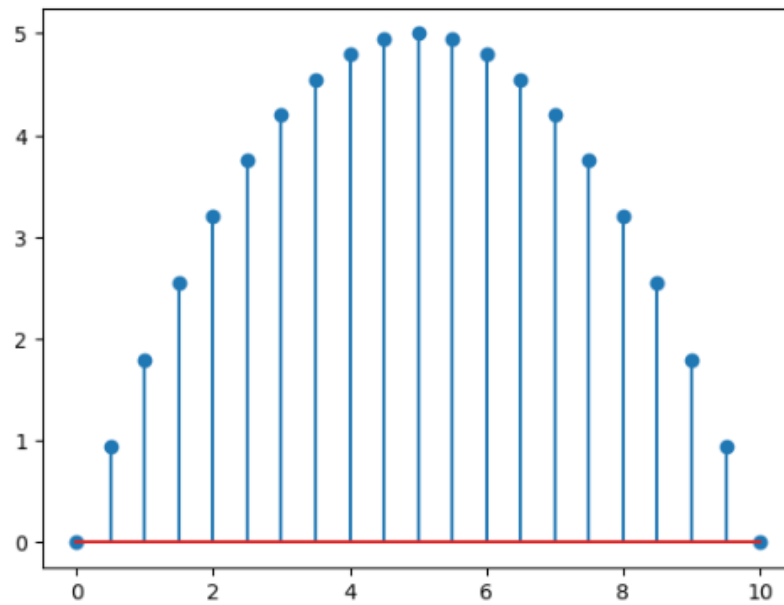


Рисунок 25. Пример 22

```
plt.stem(x, y, linefmt="r--", markerfmt="^", bottom=1)
```

<StemContainer object of 3 artists>

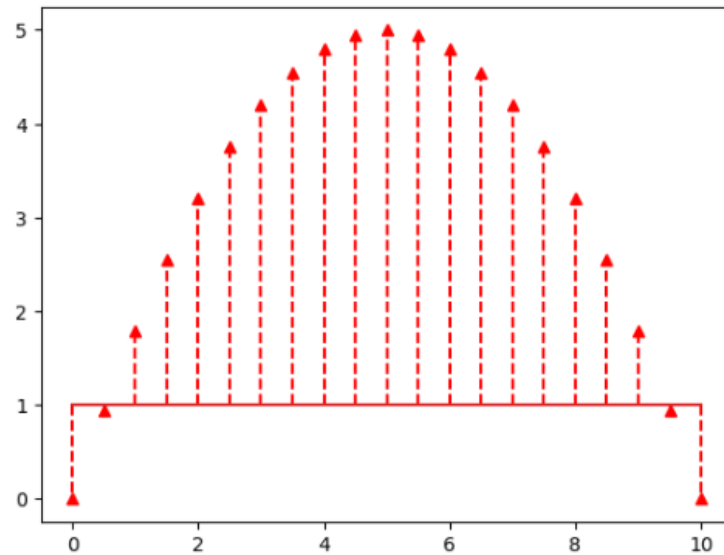


Рисунок 26. Пример 23

```
y = np.cos(x)

plt.scatter(x, y, s=80, c='r', marker='d', linewidths=2, edgecolors='g')

<matplotlib.collections.PathCollection at 0x1d15de91a90>
```

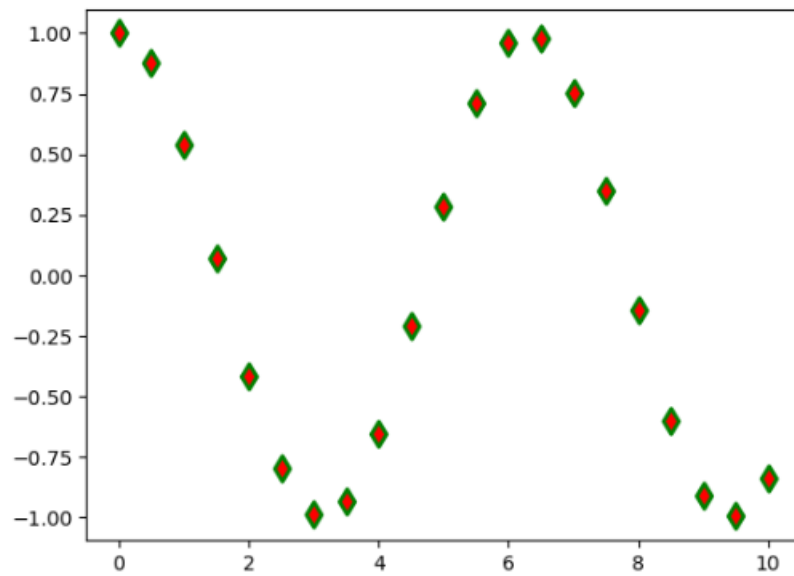


Рисунок 27. Пример 24

```
import matplotlib.colors as mcolors

bc = mcolors.BASE_COLORS

x = np.arange(0, 10.5, 0.25)
y = np.cos(x)

num_set = np.random.randint(1, len(mcolors.BASE_COLORS), len(x))
sizes = num_set * 35
colors = [list(bc.keys())[i] for i in num_set]

plt.scatter(x, y, s=sizes, alpha=0.4, c=colors, linewidths=2, edgecolors="face")

plt.plot(x, y, "g--", alpha=0.4)

[<matplotlib.lines.Line2D at 0x1d1615802d0>]
```

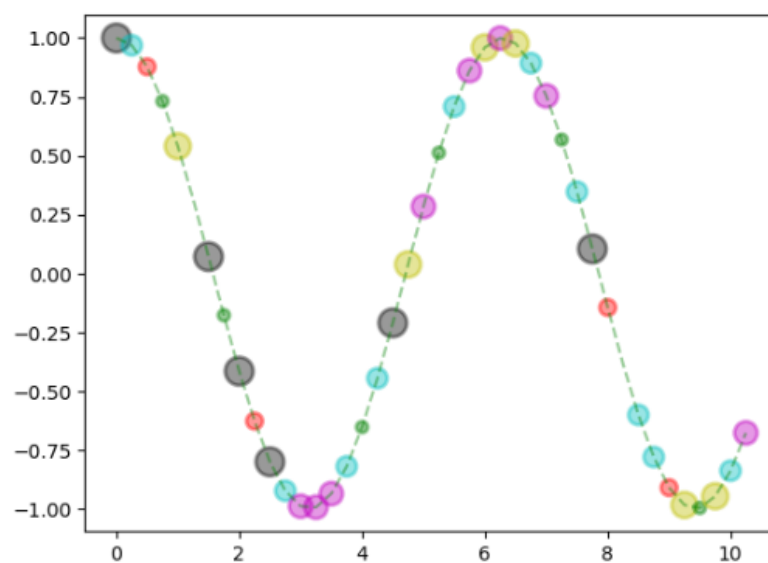


Рисунок 28. Пример 25


```
np.random.seed(123)

groups = [f"P{i}" for i in range(7)]
counts = np.random.randint(3, 10, len(groups))

plt.bar(groups, counts)
```

<BarContainer object of 7 artists>

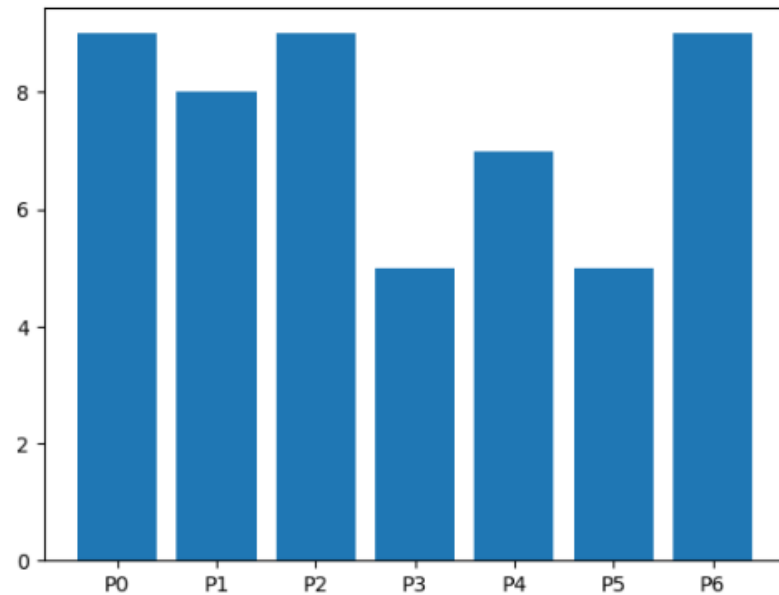


Рисунок 29. Пример 26

```
plt.barh(groups, counts)
```

<BarContainer object of 7 artists>

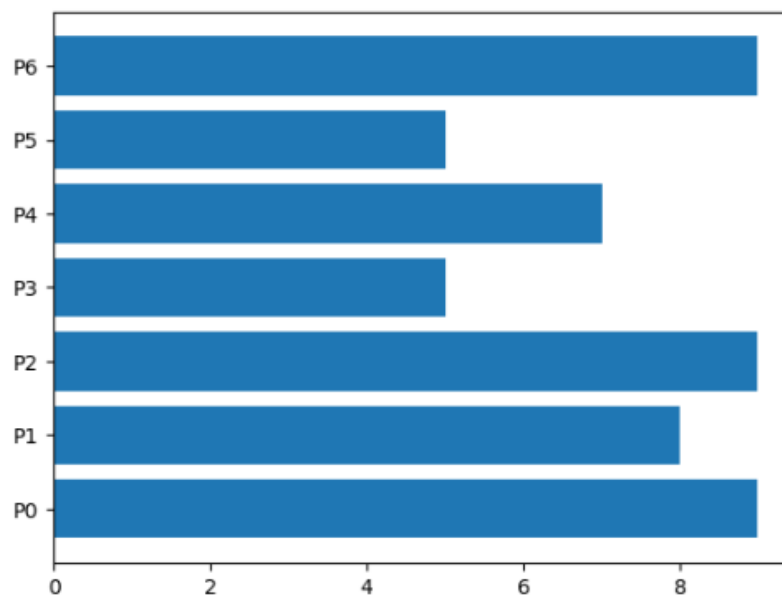


Рисунок 30. Пример 27

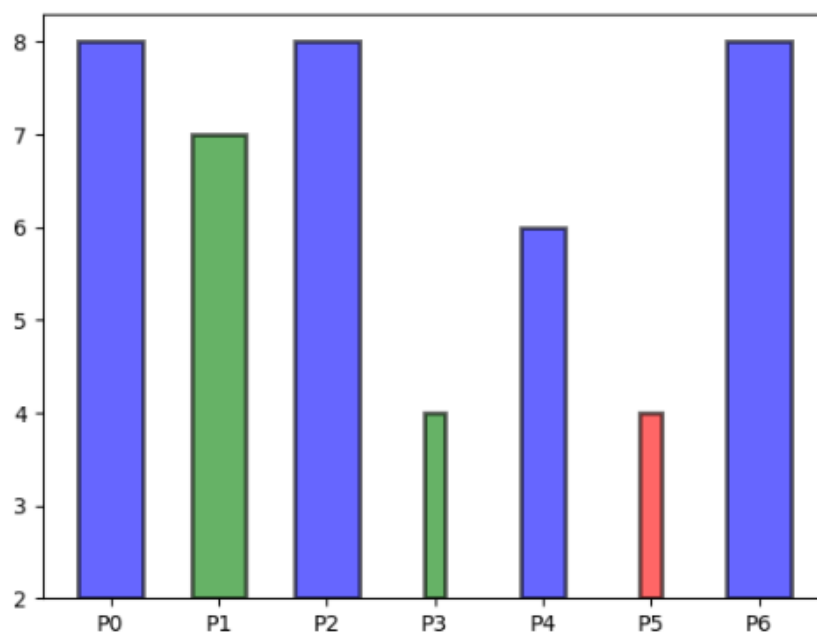


Рисунок 31. Пример 28

```
cat_par = [f"P{i}" for i in range(5)]

g1 = [10, 21, 34, 12, 27]
g2 = [17, 15, 25, 21, 26]

width = 0.3

x = np.arange(len(cat_par))

fig, ax = plt.subplots()
rects1 = ax.bar(x - width/2, g1, width, label="g1")
rects2 = ax.bar(x + width/2, g2, width, label="g2")

ax.set_title('Пример групповой диаграммы')
ax.set_xticks(x)
ax.set_xticklabels(cat_par)

ax.legend()
```

<matplotlib.legend.Legend at 0x1d161aca0d0>

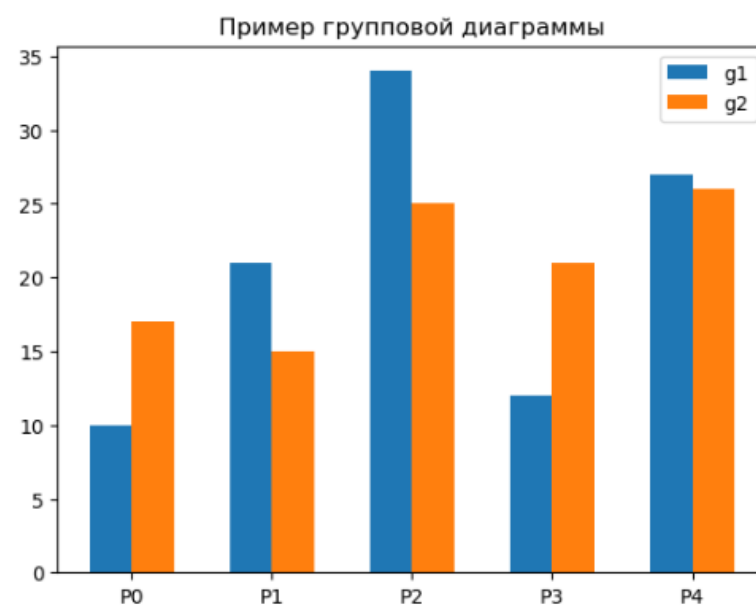


Рисунок 32. Пример 29

```

np.random.seed(123)

rnd = np.random.randint

cat_par = [f"P{i}" for i in range(5)]
g1 = [10, 21, 34, 12, 27]

error = np.array([[rnd(2, 7), rnd(2, 7)] for _ in range(len(cat_par))]).T
fig, axs = plt.subplots(1, 2, figsize=(10, 5))

axs[0].bar(cat_par, g1, yerr=5, ecolor="r", alpha=0.5, edgecolor="b", linewidth=2)
axs[1].bar(cat_par, g1, yerr=error, ecolor="r", alpha=0.5, edgecolor="b", linewidth=2)

```

<BarContainer object of 5 artists>

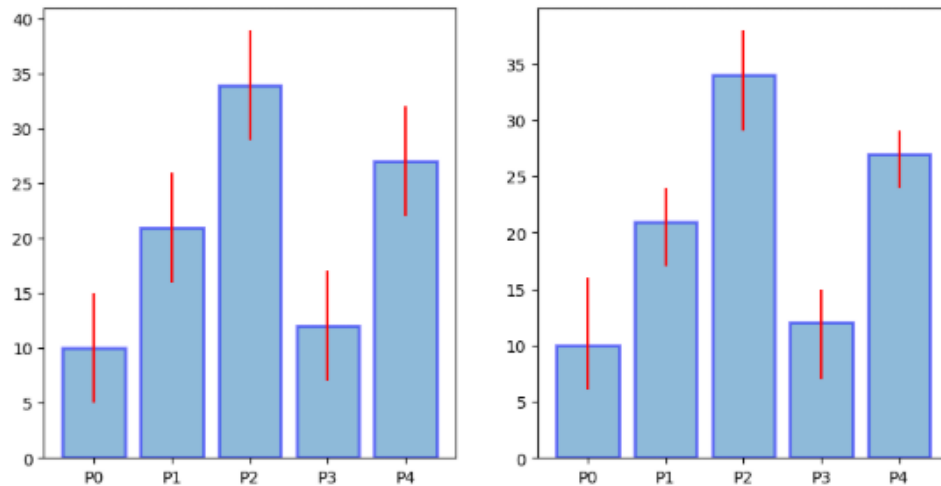


Рисунок 33. Пример 30

```

vals = [24, 17, 53, 21, 35]

labels = ["Ford", "Toyota", "BMW", "AUDI", "Jaguar"]

fig, ax = plt.subplots()
ax.pie(vals, labels=labels)
ax.axis("equal")

(np.float64(-1.099996446863133),
 np.float64(1.0999998308030063),
 np.float64(-1.099996500566958),
 np.float64(1.0999997141645375))

```

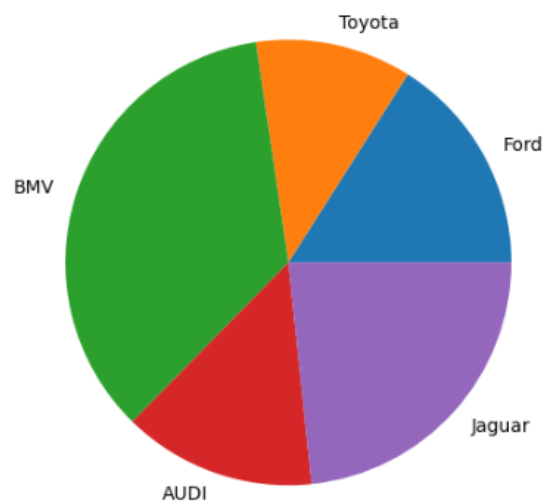


Рисунок 34. Пример 31

```
explode = (0.1, 0, 0.15, 0, 0)

fig, ax = plt.subplots()

ax.pie(vals, labels=labels, autopct='%1.1f%%', shadow=True, explode=explode, wedgepr

ax.axis("equal")

(np.float64(-1.2541693795448878),
 np.float64(1.199144954675332),
 np.float64(-1.1017809084520842),
 np.float64(1.137406140467696))
```

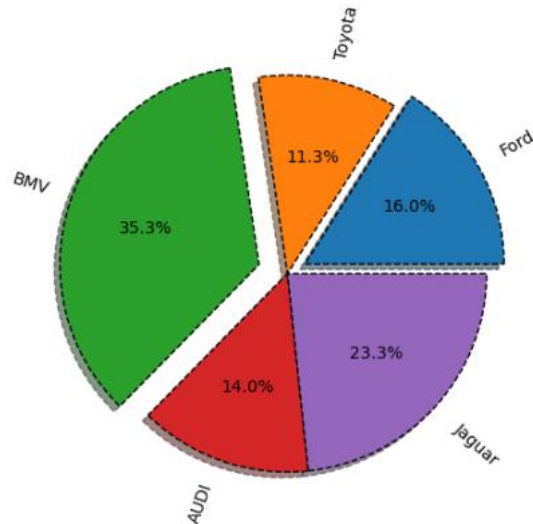


Рисунок 35. Пример 32

```
fig, ax = plt.subplots()

offset = 0.4

data = np.array([[5, 10, 7], [8, 15, 5], [11, 9, 7]])
cmap = plt.get_cmap("tab20b")

b_colors = cmap(np.array([0, 8, 12]))
sm_colors = cmap(np.array([1, 2, 3, 9, 10, 11, 13, 14, 15]))

ax.pie(data.sum(axis=1), radius=1, colors=b_colors, wedgeprops=dict(width=offset, ed

ax.pie(data.flatten(), radius=1-offset, colors=sm_colors, wedgeprops=dict(width=offs

ax.axis("equal")

(np.float64(-1.0999964890925447),
 np.float64(1.0999998328139307),
 np.float64(-1.0999994518653897),
 np.float64(1.0999989223381248))
```

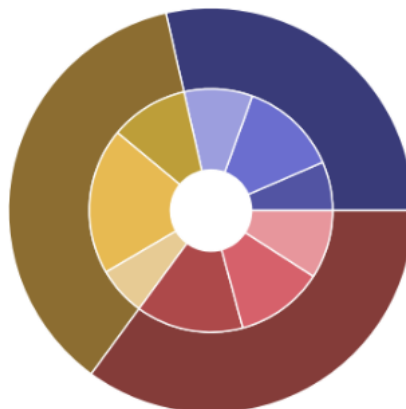


Рисунок 36. Пример 33

```
fig, ax = plt.subplots()
ax.pie(vals, labels=labels, wedgeprops=dict(width=0.5))
plt.show

<function matplotlib.pyplot.show(close=None, block=None)>
```

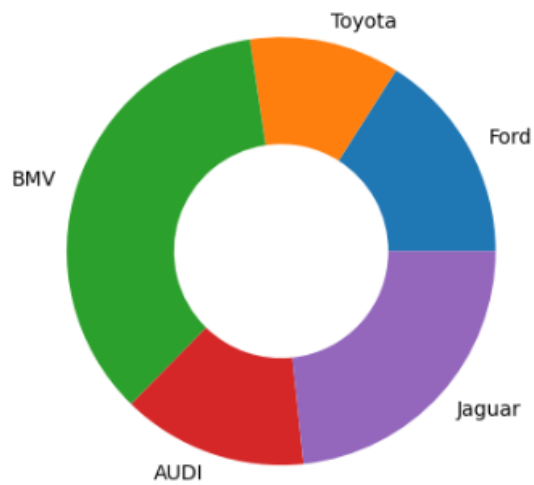


Рисунок 37. Пример 34

```
np.random.seed(19680801)
data = np.random.randn(25, 25)
plt.imshow(data)
```

<matplotlib.image.AxesImage at 0x1d15de91fd0>

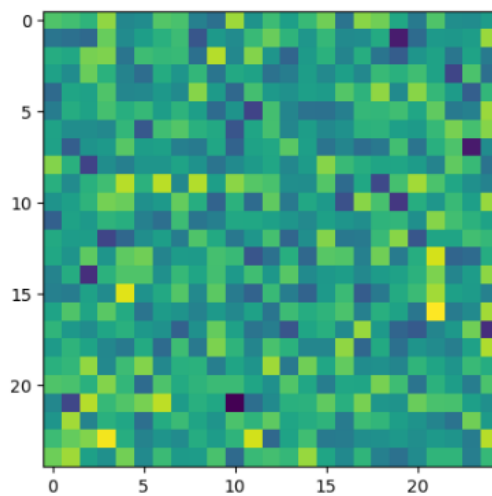


Рисунок 38. Пример 35

```
fig, axes = plt.subplots(1, 2, figsize=(10, 3), constrained_layout=True)

p1 = axes[0].imshow(data, cmap='winter', aspect='equal', vmin=-1, vmax=1, origin="low")
fig.colorbar(p1, ax=axes[0])

p2 = axes[1].imshow(data, cmap='plasma', aspect='equal', interpolation='gaussian', or
fig.colorbar(p2, ax=axes[1])
```

<matplotlib.colorbar.Colorbar at 0x1d161bace10>

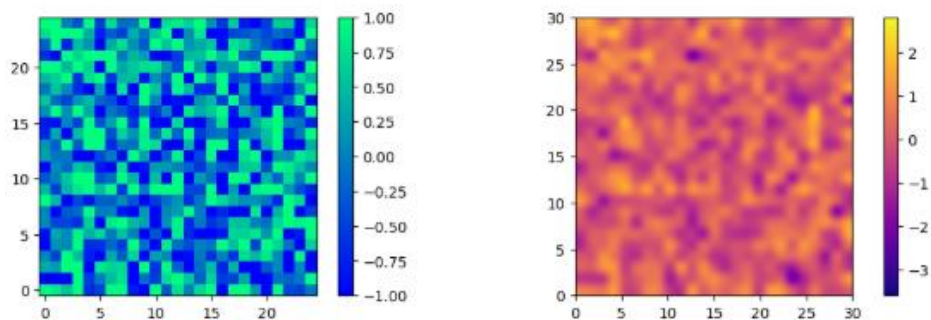


Рисунок 39. Пример 36

```
np.random.seed(123)

data = np.random.rand(5, 7)
plt.pcolormesh(data, cmap='plasma', edgecolors='k', shading='flat')
```

<matplotlib.collections.QuadMesh at 0x1d161d49f90>

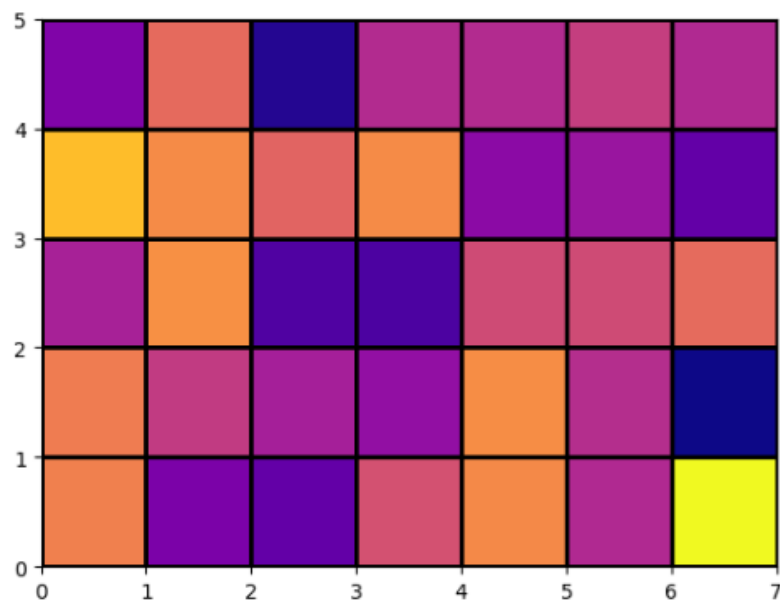


Рисунок 40. Пример 37

```
x = np.linspace(-np.pi, np.pi, 50)
y = x
z = np.cos(x)

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot(x, y, z, label='parametric curve')
```

[<mpl_toolkits.mplot3d.art3d.Line3D at 0x1d162f10690>]

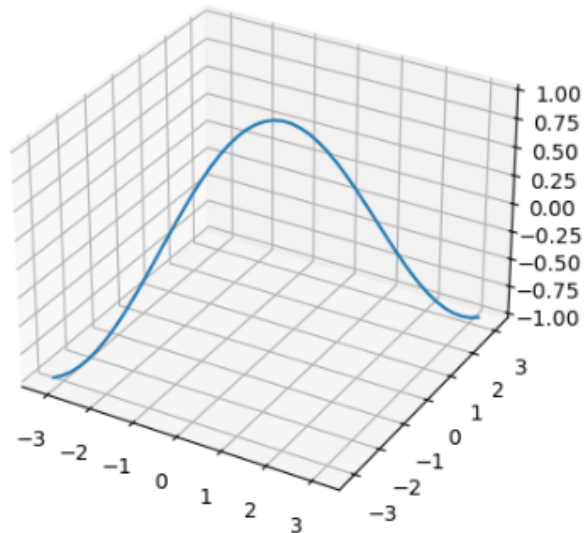


Рисунок 41. Пример 38

```
np.random.seed(123)
x = np.random.randint(-5, 5, 40)
y = np.random.randint(0, 10, 40)
z = np.random.randint(-5, 5, 40)
s = np.random.randint(10, 100, 40)

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.scatter(x, y, z, s=s)
```

<mpl_toolkits.mplot3d.art3d.Path3DCollection at 0x1d15fbfac10>

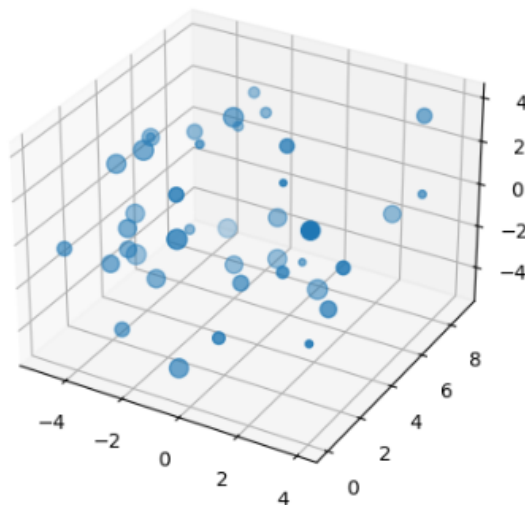


Рисунок 42. Пример 39

```

u, v = np.mgrid[0:2*np.pi:20j, 0:np.pi:10j]
x = np.cos(u)*np.sin(v)
y = np.sin(u)*np.sin(v)
z = np.cos(v)

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_wireframe(x, y, z)
ax.legend()

```

C:\Users\USER\AppData\Local\Temp\ipykernel_14576\3792594722.py:9: UserWarning: The following artists with labels found to put in legend. Note that artists whose labels contain an underscore are ignored when legend() is called with no argument.

```

ax.legend()
<matplotlib.legend.Legend at 0x1d16020bd90>

```

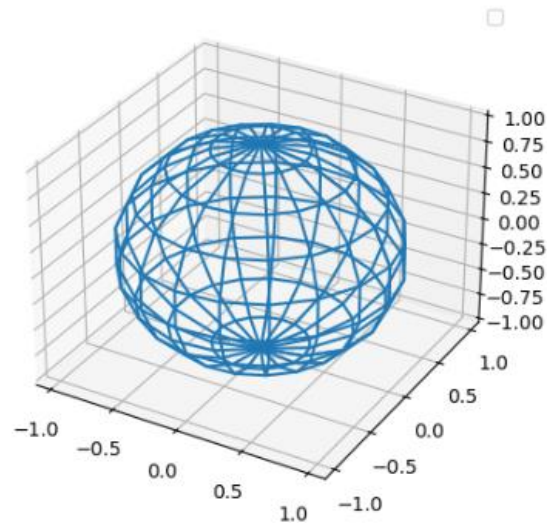


Рисунок 43. Пример 40


```

u, v = np.mgrid[0:2*np.pi:20j, 0:np.pi:10j]
x = np.cos(u)*np.sin(v)
y = np.sin(u)*np.sin(v)
z = np.cos(v)

```

```

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(x, y, z, cmap='inferno')
ax.legend()

```

C:\Users\USER\AppData\Local\Temp\ipykernel_14576\3126093335.py:9: Artists with labels found to put in legend. Note that artists whose an underscore are ignored when legend() is called with no argument

ax.legend()

<matplotlib.legend.Legend at 0x1d15f7e3d90>

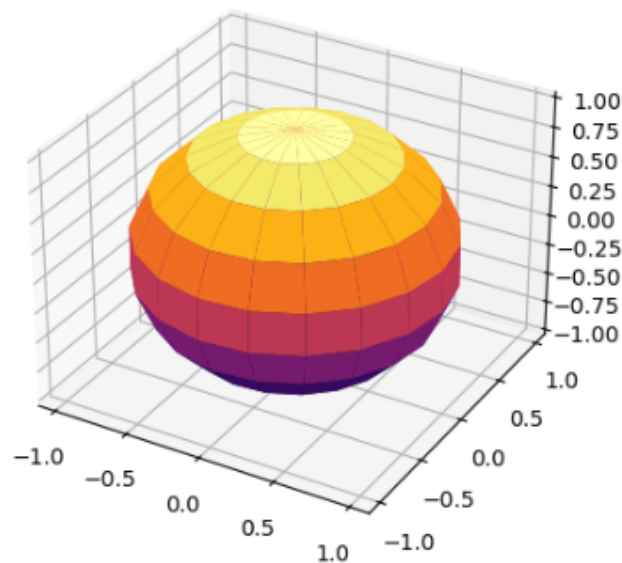


Рисунок 44. Пример 41

5) Были выполнены задания по своему варианту (вариант 17).

Задание 1: Напишите код, который строит график функции $y = x^2$ на интервале $[-10, 10]$. Добавьте заголовок, подписи осей и сетку.

```
import matplotlib.pyplot as plt
import numpy as np
%matplotlib inline

x = np.arange(-10, 11, 1)
y = x ** 2

plt.title("График функции  $y = x^2$ ")
plt.xlabel("x")
plt.ylabel("y")
plt.grid()

plt.plot(x, y)
```

[<matplotlib.lines.Line2D at 0x201740bb750>]

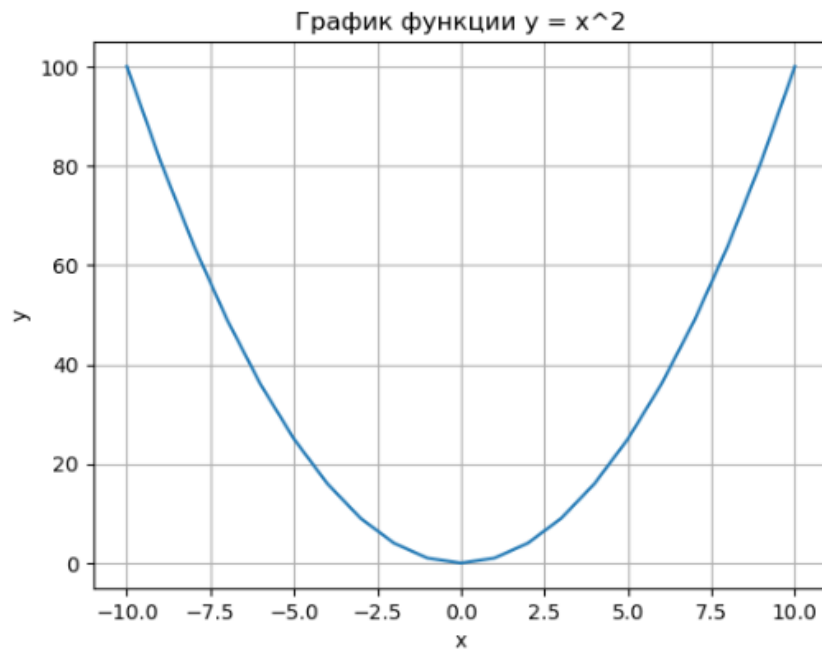


Рисунок 45. Задание 1

Задание 2: Постройте три линии на одном графике: $y = x$ (синяя, пунктирная линия), $y = x^2$ (зеленая, штрих-пунктирная линия), $y = x^3$ (красная, сплошная линия). Добавьте легенду и сделайте оси одинакового масштаба

```
[3]: x = np.linspace(-2, 2, 50)

y1 = x
y2 = x**2
y3 = x**3

plt.plot(x, y1, 'b--', label='y = x')
plt.plot(x, y2, 'g-.', label='y = x^2')
plt.plot(x, y3, 'r-', label='y = x^3')

plt.legend()

plt.axis('equal')

[3]: (np.float64(-2.2), np.float64(2.2), np.float64(-8.8), np.float64(8.8))
```

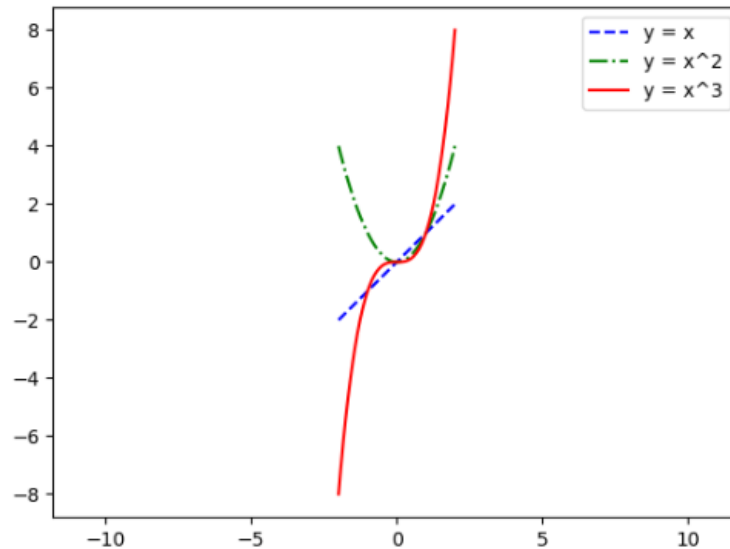


Рисунок 46. Задание 2

Задание 3: Сгенерируйте 50 случайных точек и постройте диаграмму рассеяния (scatter plot), где цвет точек зависит от их координаты по оси x , а размер точек зависит от координаты по оси y .

```

np.random.seed(67)
x = np.random.rand(50)
y = np.random.rand(50)

plt.scatter(x, y, c=x, s=y * 10)
plt.colorbar()
plt.show()

```

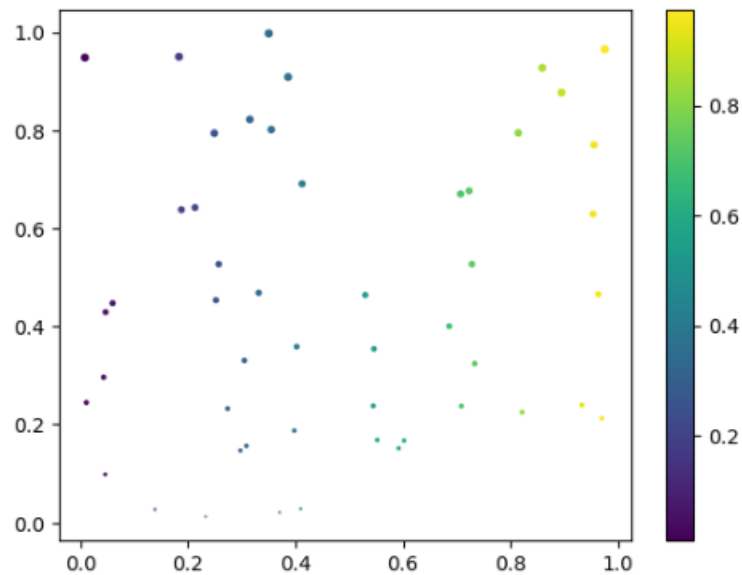


Рисунок 47. Задание 3

Задание 4: Сгенерируйте 1000 случайных чисел из нормального распределения с параметрами $\mu = 0$, $\sigma = 1$ и постройте их гистограмму с 30 бинами. Добавьте вертикальную линию в среднем значении.

```
data = np.random.normal(0, 1, 1000)

plt.hist(data, bins=30, edgecolor='black', alpha=0.7)

mean_val = np.mean(data)
plt.axvline(mean_val, color='red', linestyle='--', linewidth=2, label=f'Среднее = {mean_val}')

plt.xlabel('Значение')
plt.ylabel('Частота')
plt.title('Гистограмма нормального распределения (1000 точек)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

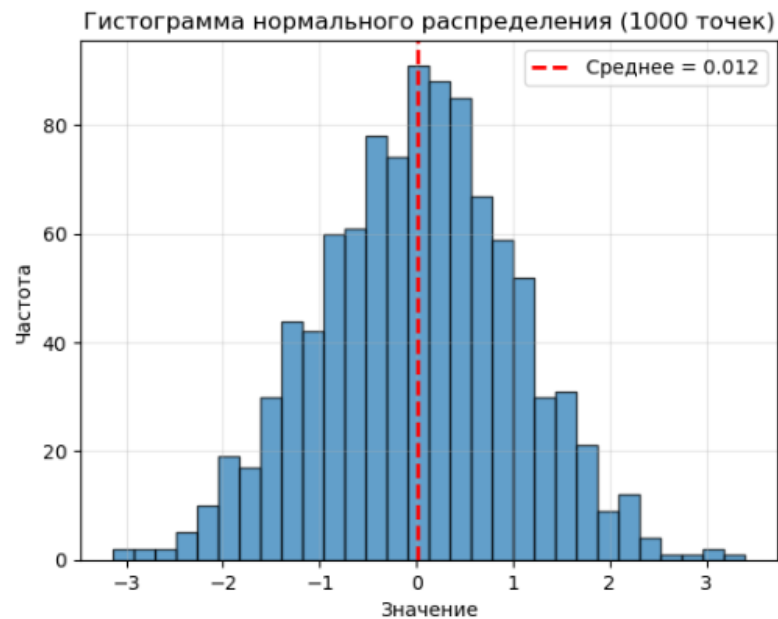


Рисунок 48. Задание 4

Задание 5: Создайте столбчатую диаграмму, которая показывает количество студентов, получивших оценки:

"Отлично" - 20 человек,

"Хорошо" - 35 человек

"Удовлетворительно" - 30 человек,

"Неудовлетворительно" - 15 человек.

Добавьте подписи к осям и заголовок.

```
students = [20, 35, 30, 15]
marks = ["Отлично", "Хорошо", "Удовлетворительно", "Неудовлетворительно"]

plt.figure(figsize = ( 9, 7))
plt.bar(marks, students)
plt.title("Оценки!")
plt.xlabel("Оценки")
plt.ylabel("Количество студентов")
```

```
Text(0, 0.5, 'Количество студентов')
```

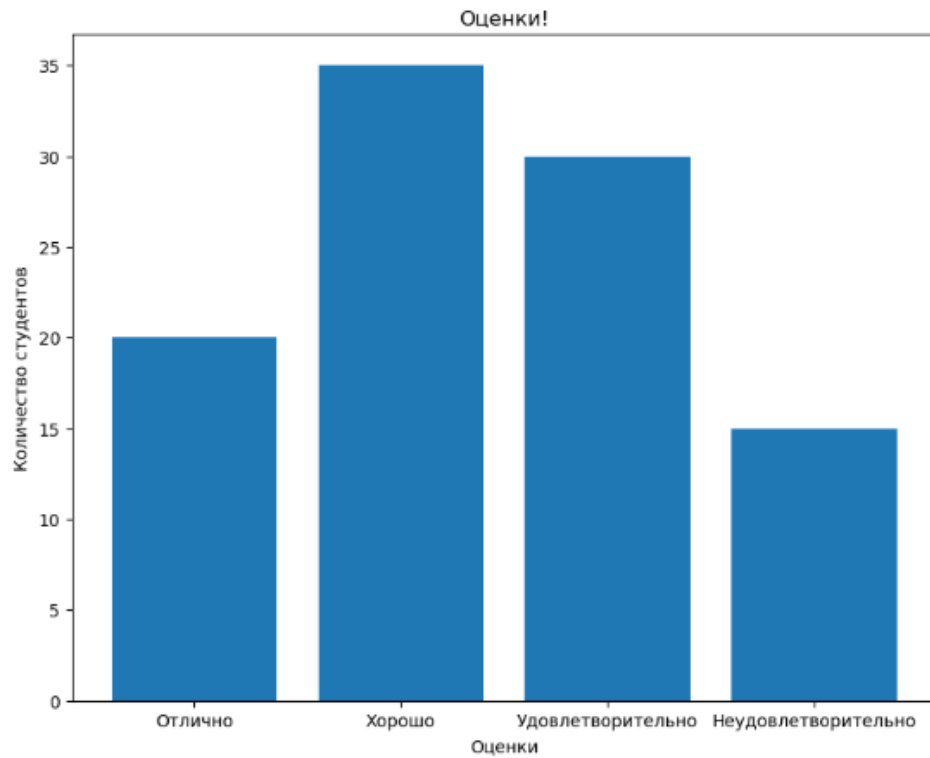


Рисунок 49. Задание 5

Задание 6: Используя данные предыдущей задачи, постройте круговую диаграмму с процентными подписями секторов

```
plt.pie(students, labels=marks, autopct='%1.1f%%')  
plt.axis('equal')  
plt.show()
```



Рисунок 50. Задание 6

Задание 7: Используя `mpl_toolkits.mplot3d`, постройте 3-D график функции $z = \sin(\sqrt{x^2 + y^2})$ на сетке значений x, y в диапазоне $[-5, 5]$.

```

fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

surf = ax.plot_surface(X, Y, Z, edgecolor='none', alpha=0.9)

ax.set_title('3D график функции $z = \sqrt{x^2 + y^2}$')
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')

plt.show()

```

3D график функции $z = \sqrt{x^2 + y^2}$

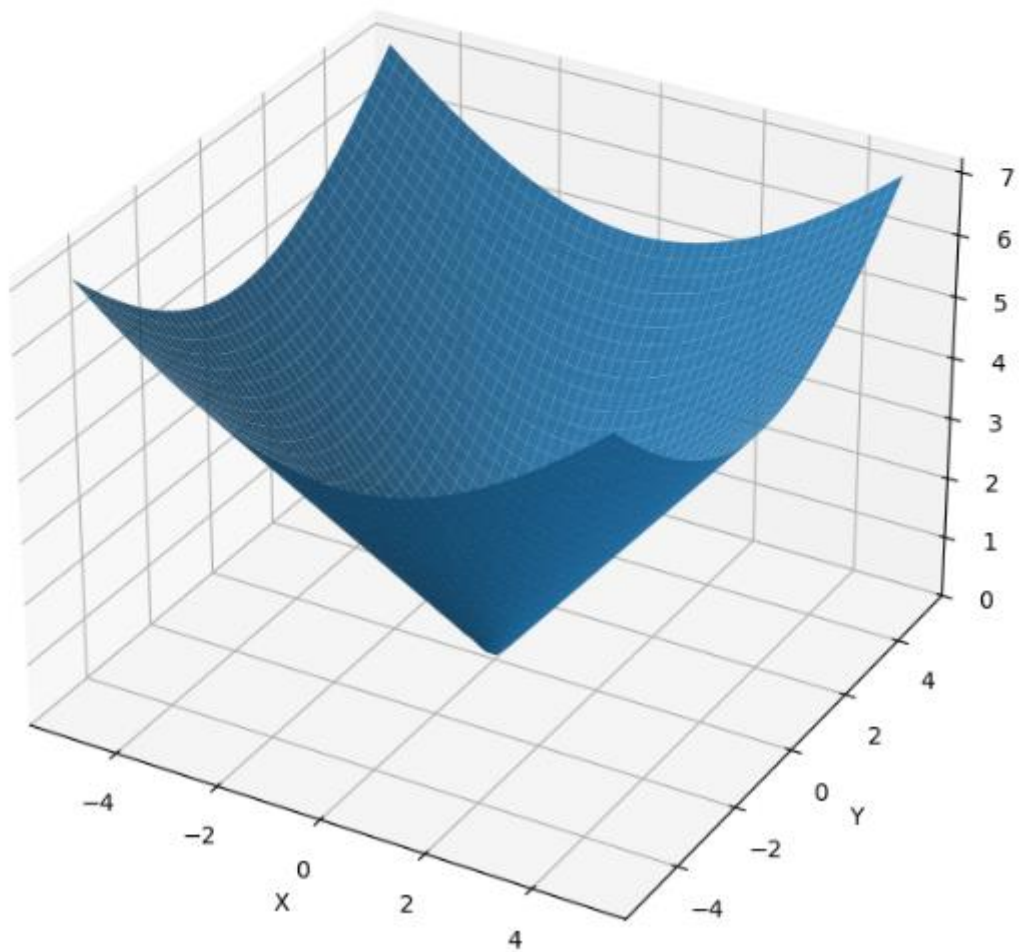


Рисунок 51. Задание 7

Задание 8: Постройте четыре графика в одной фигуре (2 x 2 сетка):
 Линейный график, парабола, синус, косинус
 Добавьте заголовки к каждому подграфику.


```
plt.xlabel("x")
plt.ylabel("y")

plt.subplot(2, 2, 3)
plt.plot(x, np.sin(x))
plt.title("y = sin(x)")
plt.xlabel("x")
plt.ylabel("y")

plt.subplot(2, 2, 4)
plt.plot(x, np.cos(x))
plt.title("y = cos(x)")
plt.xlabel("x")
plt.ylabel("y")
```

[34]: Text(0, 0.5, 'y')

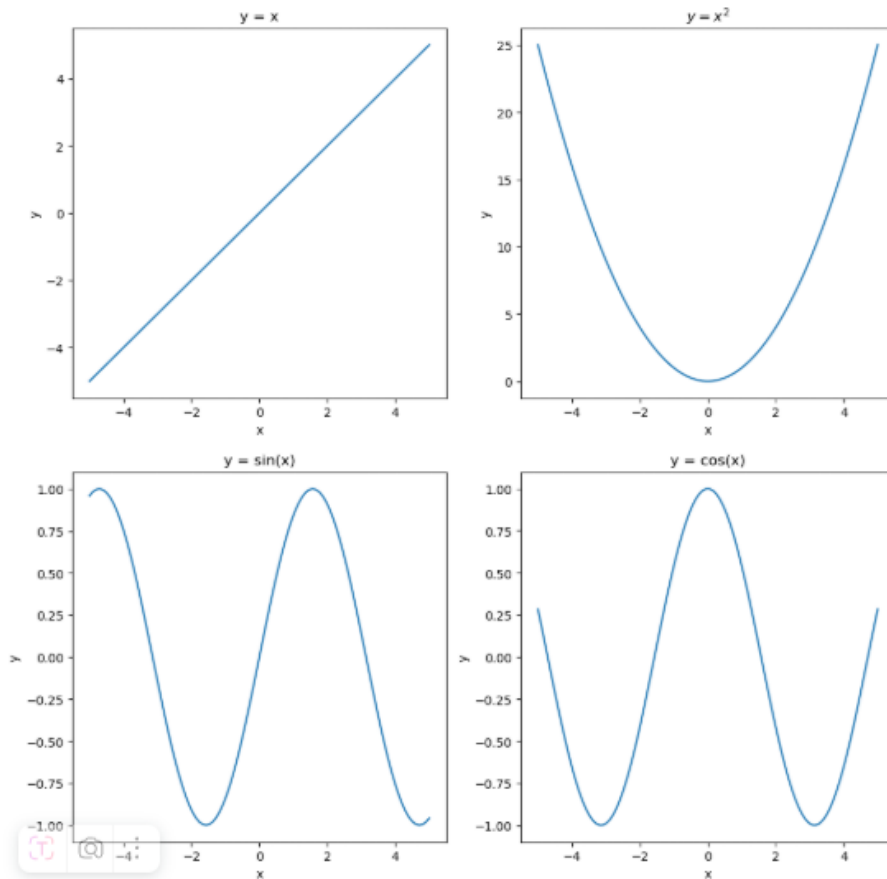


Рисунок 52. Задание 8

Задание 9: Создайте случайную матрицу 10 x 10 с элементами от 0 до 1 и визуализируйте ее как тепловую карту с цветовой шкалой.

```
matrix = np.random.rand(10, 10)

plt.figure(figsize=(8, 6))
plt.imshow(matrix, cmap='magma', interpolation='nearest', vmin=0, vmax=1)
plt.colorbar(label='Значение')
plt.title('Тепловая карта случайной матрицы 10×10')
plt.xlabel('Столбцы')
plt.ylabel('Строки')
plt.show()
```

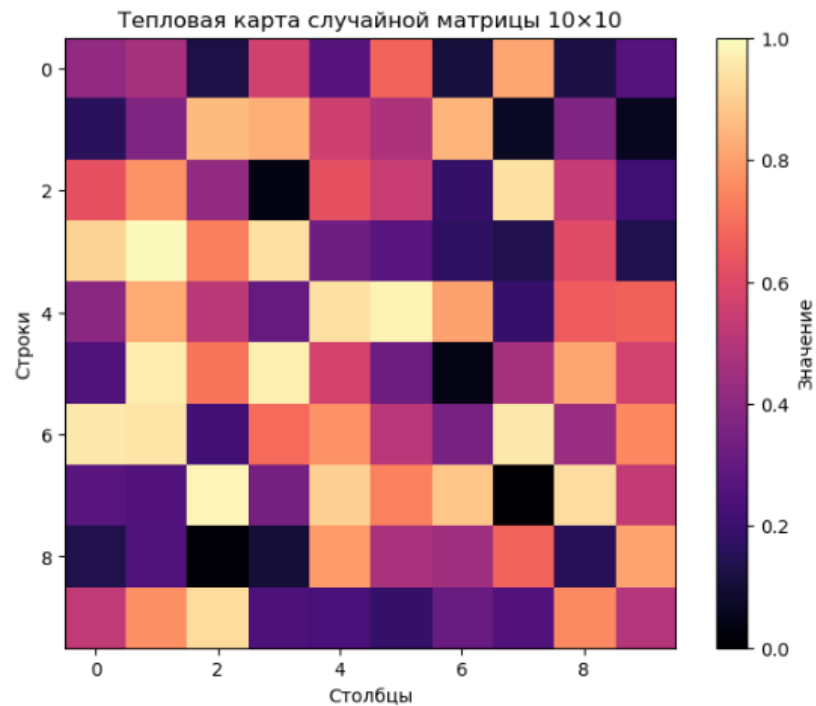


Рисунок 53. Задание 9

Индивидуальное задание 1: Решите систему линейных уравнений вида:

За год фиксировались среднемесячные осадки в мм:

Месяцы: ["Янв", "Фев", "Март", "Апр", "Май", "Июнь", "Июль", "Авг", "Сен", "Окт", "Нояб", "Дек"]

Осадки (мм): [45, 38, 50, 60, 70, 90, 110, 95, 80, 60, 55, 48]

Используйте два разных цвета для осадков выше и ниже среднего значения.

```
months = ["Янв", "Фев", "Март", "Апр", "Май", "Июнь", "Июль", "Авг", "Сен", "Окт", "Нояб", "Дек"]
values = [45, 38, 50, 60, 70, 90, 110, 95, 80, 60, 55, 48]

mean_v = np.mean(values)
colors = ['red' if v > mean_v else 'blue' for v in values]

plt.scatter(months, values, c=colors)
plt.axhline(mean_v, color='green', linestyle='--', label=f'Среднее = {mean_v:.1f}')
plt.legend()
plt.grid()
plt.title('Осадки по месяцам')
plt.show()
```

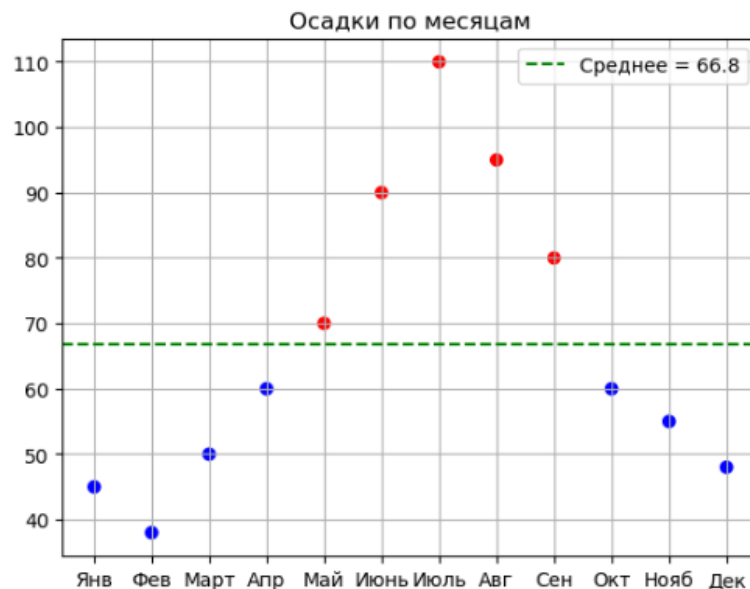


Рисунок 54. Индивидуальное задание 1

Индивидуальное задание 2: Данные о средней продолжительности жизни (в годах):

Страны: ["Япония", "Швейцария", "Австралия", "Швеция", "США"]

Возраст: [84, 83, 82, 81, 78]

Используйте разный цвет столбцов для стран с продолжительностью жизни выше и ниже 80 лет

```

countries = ["Япония", "Швейцария", "Австралия", "Швеция", "США"]
ages = [84, 83, 82, 81, 78]

colors = ['green' if age >= 80 else 'orange' for age in ages]

plt.figure(figsize=(10, 8))
plt.bar(countries, ages, color=colors)

plt.axhline(y=80, color='red', linestyle='--', label='Порог 80 лет')

plt.title('Средняя продолжительность жизни в странах')
plt.xlabel('Страны')
plt.ylabel('Возраст (лет)')
plt.grid(axis='y', linestyle=':', alpha=0.7)
plt.legend()
plt.show()

```

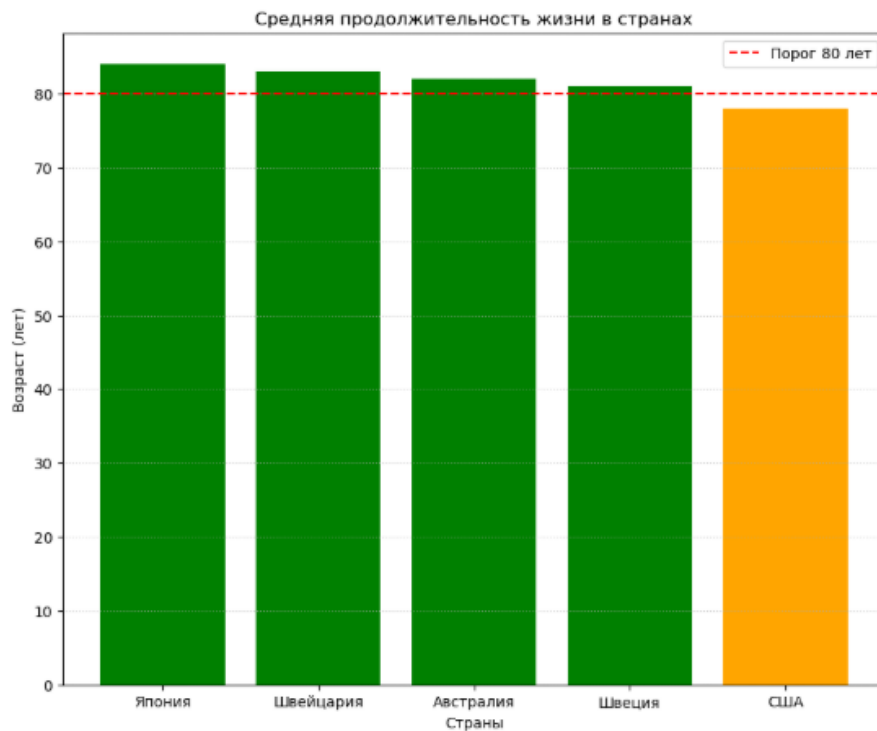


Рисунок 55. Индивидуальное задание 2

Индивидуальное задание 3: Площадь под логарифмической функцией с коэффициентом

1. Построить график подынтегральной функции.
2. Вычислить площадь под кривой на заданном отрезке как значение определенного интеграла

3. Закрасить область под графиком, чтобы визуализировать интеграл.

Вычислите интеграл $f(x) = 2\ln(x) + x$ на отрезке $[1, 3]$.

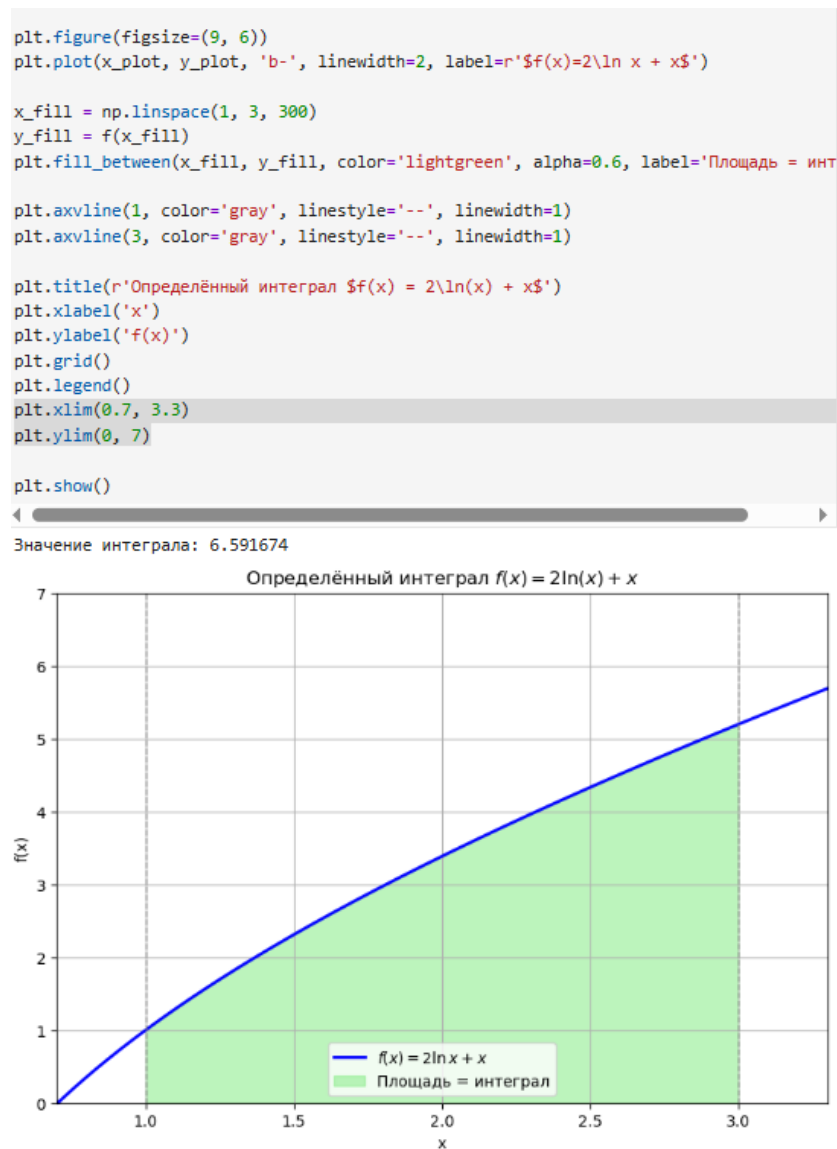


Рисунок 56. Индивидуальное задание 3

Индивидуальное задание 4: Спираль Архимеда в 3D

1. Построить трехмерный график функции $f(x, y)$ в заданных пределах.
2. Использовать библиотеку `matplotlib` для визуализации.
3. Оформить график: добавить заголовок, подписи осей и цветовую карту (если уместно).

Построить параметрическую спираль: $x = t\cos(t)$, $y = t\sin(t)$, $z = t$, $t \in [0, 4\pi]$.

```

t = np.linspace(0, 4 * np.pi, 1000)

x = t * np.cos(t)
y = t * np.sin(t)
z = t

fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(x, y, z, c=t, cmap='summer', s=5, alpha=0.7)

ax.set_title('Спираль Архимеда в 3D:  $x = t \cos t$ ,  $y = t \sin t$ ,  $z = t$ ', fontsize=14)
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')

cbar = fig.colorbar(scatter, ax=ax, shrink=0.5, aspect=20)
cbar.set_label('Параметр t', fontsize=10)

plt.show()

```

Спираль Архимеда в 3D: $x = t \cos t$, $y = t \sin t$, $z = t$

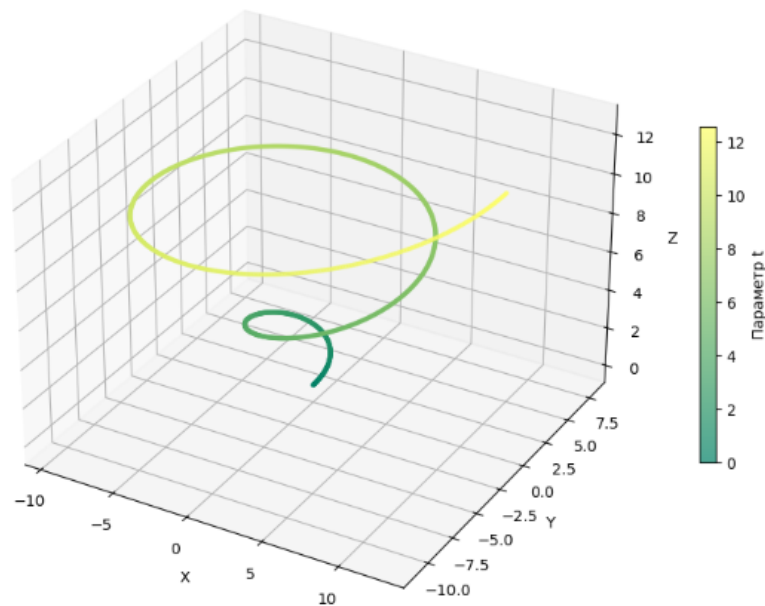


Рисунок 57. Индивидуальное задание 4

Ответы на контрольные вопросы:

- 1) Существует два основных варианта установки библиотеки matplotlib: в первом случае вы устанавливаете пакет Anaconda, в состав которого входит большое количество различных инструментов для работы в области машинного обучения и анализа данных (и не только); во втором – установить Matplotlib самостоятельно, используя менеджер пакетов.
- 2) Для корректного отображения графиков matplotlib в ноутбуках Jupyter должна присутствовать магическая команда `%matplotlib inline`.
- 3) Для отображения графика с помощью функции plot нужно ввести `plt.plot()`

4) Для того, чтобы построить несколько графиков на одном поле в функцию `plot` нужно последовательно передать два массива для построения первого графика и два массива для построения второго.

5) Для построения диаграмм категориальных данных нужно воспользоваться функцией `bar()`.

6) Основные элементы графика: фигура, основная и дополнительная сетка, основные и дополнительные тики, заголовок (`title`), легенда (`legend`), подписи осей `x` и `y`, линейный и точечный графики.

7) Наиболее часто используемые текстовые надписи на графике это: наименование осей, наименование самого графика, текстовое примечание на поле с графиком, легенда. Для задания подписей осей используются функции `xlabel` и `ylabel`. Для задания заголовка графика используется функция `title()`. Для отображения текстового примечания используется функция `text()`.

8) Для отображения легенды используется функция `legend()`, а в функции `plot` в качестве одного из аргументов надо передать `label=""`.

9) Для задания цвета и стиля линий можно передать сразу после указания списков с координатами в кавычках первую букву цвета из набора символов `{b, g, r, c, m, y, k, w}` и тип линии `{-, --, -., .}`. Их можно задавать используя слова `color` и `linestyle` или без их использования. Также можно модифицировать через функцию `setp`.

10) Для того, чтобы разместить графики на разных полях есть три способа: использование функции `subplot()` для указания места размещения поля с графиком; использование функции `subplots()` для предварительного задания сетки, в которую будут укладываться поля; использование функции `GridSpec`, для более гибкого задания геометрии размещения полей с графиками в сетке.

11) Для построения линейного графика нужно воспользоваться функцией `plot()`: `plt.plot(x, y)`

12) Заливка:

Между графиком и осью: `plt.fill_between(x, y, 0)`

Между двумя графиками: `plt.fill_between(x, y1, y2)`

13) Чтобы выполнить выборочную заливку, которая удовлетворяет некоторому условию нужно использовать параметр `where` в `fill_between`, которому присвоить переменную с выполнением условия:

```
condition = (y > 0)
```

```
plt.fill_between(x, y, 0, where=condition, color='green', alpha=0.5)
```

14) Двухцветная заливка:

```
plt.fill_between(x, y, 0, where=(y >= 0), color='green', alpha=0.5)
```

```
plt.fill_between(x, y, 0, where=(y < 0), color='red', alpha=0.5)
```

15) Чтобы выполнить маркировку графиков нужно воспользоваться параметром `marker`: `plt.plot(x, y, marker='o', markersize=6)`.

16) Обрезка графика выполняется через: `plt.xlim(left, right)`, `plt.ylim(bottom, top)`

17) Ступенчатый график строится с помощью функции `step()`: `plt.step(x, y, where='pre')`. Особенность: значения остаются постоянными между последовательными точками, создавая "ступеньки".

18) Для построения стекового графика используется функция `stackplot()`. Суть его в том, что графики отображаются друг над другом, и каждый следующий является суммой предыдущего и заданного набора данных:

```
fig, ax = plt.subplots()
```

```
ax.stackplot(x, y1, y2, y3, labels=labels)
```

19) Чтобы построить `stem`-график нужно воспользоваться функцией `stem()`: `plt.stem(x, y)`. Визуально этот график выглядит как набор линий от точки с координатами (x, y) до базовой линии, в верхней точке ставится маркер.

20) Для построения точечного графика предназначена функция `scatter()`. Особенность: каждая точка может иметь свой размер (`s`), цвет (`c`) и непрозрачность. Позволяет визуализировать трёхмерные данные (X , Y , цвет/размер) на плоскости. В отличие от `plot` с

маркерами, scatter масштабируем для больших данных и даёт контроль над каждой точкой.

21) Для построения столбчатых диаграмм используются функции `bar()`, `barh()`.

22) Групповая столбчатая диаграмма – несколько рядов столбцов, сгруппированных по категориям. Реализуется путём сдвига позиций (например, $x - \text{width}/2$, $x + \text{width}/2$ для двух рядов)

Столбчатая диаграмма с `errorbar` – к каждому столбцу добавляется вертикальная (или горизонтальная) планка, показывающая разброс/стандартную ошибку/доверительный интервал. Параметр `err` (или `herr` для горизонтальных)

23) Для построения круговых диаграмм в Matplotlib используется функция `pie()`:

```
fig, ax = plt.subplots()
ax.pie(vals, labels = labels)
```

24) Цветовая карта представляет собой подготовленный набор цветов, который хорошо подходит для визуализации того или иного набора данных. Получить карту: `cmmap = plt.cm.viridis`. Можно использовать в `scatter` (параметр `c`, затем `cmmap`), `contourf`, `imshow`, `pcolormesh`.

25) Отображение изображения:

```
img = mpimg.imread('photo.jpg')
plt.imshow(img)
```

26) Тепловая карта – это раскрашенная матрица. Делается через `imshow` или `pcolormesh`: `plt.imshow(data, cmap="hot", interpolation="nearest")`.

27) Для построения линейного 3D-графика нужно импортировать `3D-axes`:

```
from mpl_toolkits.mplot3d import Axes3D
ax = fig.add_subplot(111, projection="3d")
ax.plot(x, y, z)
```

28) Построение точечного графика: `ax.scatter(x, y, z)`

29) Каркасная поверхность: `ax.plot_wireframe(X, Y, Z, rstride=1, cstride=1)`

30) Трехмерная поверхность: `ax.plot_surface(X, Y, Z, cmap='coolwarm')`

Вывод: в ходе работы были исследованы базовые возможности библиотеки Matplotlib языка программирования Python.